

指令缓存单元（ICU） Design Spec

状态：基于RTL重新生成

范围：Cortex-M7 r1p2 的 Instruction Cache Unit (ICU，指令缓存单元) Register Transfer Level (RTL，寄存器传输级) 实现，包括 `cm7icu`、`cm7icu_ecc_check`，以及它与 `cm7pfu`、`cm7biu`、Instruction cache (I-cache，指令缓存) Random Access Memory (RAM，随机存取存储器)、`cm7ppb` 和 `cm7miu` 的行为边界。本文是reimplementation spec：工程师应能仅依据本文实现一个端口、时序、状态、副作用和异常路径兼容的ICU，而不需要回到RTL猜测设计意图。

本文所说的“I-cache相关内容”包括：容量与地址映射、2路组相联组织、tag/data RAM格式、三种lookup、命中与缺失、Linefill Buffer (LFB，行填充缓冲)、critical-word-first前递、替换way选择、allocation、cache enable与memory attribute、invalidate维护、Error Correction Code (ECC，错误纠正码)、错误寄存器、RAM仲裁、Memory Built-In Self-Test (MBIST，存储器内建自测试)以及复位后的软件使用约束。ICU之外的Memory Protection Unit属性生成和Bus Interface Unit (BIU，总线接口单元)内部Advanced eXtensible Interface (AXI)实现只描述接口契约，不在本文重写其内部逻辑。

1. 概述与宏观架构

1.0 术语

本文反复使用以下术语。`Ic1` / `Ic2` 是 RTL 中的 pipeline stage 名，当前可读源码没有展开其英文全称；本文按功能解释这两个 stage。

Term	Meaning
Instruction Cache Unit (ICU)	指令缓存单元。接收PFU的64位取指请求，访问I-cache，缺失时向BIU请求linefill，并把指令数据或错误返回PFU。
Prefetch Unit (PFU)	指令预取单元，向 ICU 发送 fetch address，并消费 ICU 返回的 64-bit instruction data。
Bus Interface Unit (BIU)	总线接口单元，代表 ICU 到外部 instruction memory path 发起 linefill/read request，并返回 64-bit beat。
Data Processing Unit (DPU)	处理器执行/控制侧单元。ICU 端口名里部分 <code>dpu_icu_*</code> 实际由 PPB 集成过来，用于 I-cache enable、cache maintenance 和错误寄存器访问。
Private Peripheral Bus (PPB)	Cortex-M 私有外设总线侧寄存器模块，提供 Cache Control Register、Cache Auxiliary Control Register、cache maintenance 和 Error Bank Register (EBR) 访问。
Memory Built-In Self-Test (MBIST)	内建存储测试机制。ICU 通过 MIU 接收 MBIST lock/read/write 请求，并把 I-cache RAM 暴露给测试路径。
MBIST Interface Unit (MIU)	MBIST 接口单元，可锁住 ICU 并直接读写 I-cache tag/data RAM。
MBISTALL	MIU 的测试模式，表示同一 read/write transaction 同时作用到全部 I-cache arrays；read data 仍只从 selected array 返回。

Term	Meaning
I-cache	Instruction cache, ICU 管理的 2-way set-associative 指令缓存。每条 cache line 为 32 bytes。
cache hit	当前 fetch address 在 I-cache 中找到 valid tag match, 可以直接从 cache RAM 返回 instruction data。
cache miss	当前 fetch address 没有在 I-cache 中找到可用数据, 需要通过 BIU 从 instruction memory 取回。
cache line	一次 linefill 填充和一次 tag 管理的最小缓存单位, ICU 中为 4 个 64-bit doubleword。
set	I-cache 中由 index 选中的一组候选 cache line; Cortex-M7 ICU 每个 set 有两个 way。
way	set-associative cache 中同一个 set 内的候选位置; ICU 使用 way0 和 way1。
tag	cache line 保存的高位地址字段, 用于判断某个 way 是否保存了当前 fetch address 对应的 line。
index	fetch address 中用于选择 cache set 的字段; ICU 用它访问 tag/data RAM。
offset	fetch address 中用于选择 cache line 内具体 doubleword 的字段。
allocation	cache miss 后, 把 BIU/LFB 收齐的 cache line 写入 I-cache RAM 的动作。
replacement way	allocation 时选择写入 way0 还是 way1 的结果; ICU 会避开被 IEBR/ECC block 的 way。
cacheable	memory attribute 中表示该 fetch 可以使用 I-cache 并允许 linefill allocation 的属性。
doubleword	64-bit 数据单位。ICU 每次向 PFU 返回一个 doubleword instruction data。
quadword	本文指 128-bit 数据组, 即 2 个相邻 64-bit doubleword; ICU allocation 分 lower/upper 两个 quadword 写入 data RAM。
Ic1 stage	ICU fetch address/lookup request stage。接收 PFU 请求, 决定是否访问 tag/data RAM, 生成 RAM enable/address。
Ic2 stage	ICU fetch response/hit-miss stage。保存当前 fetch 上下文, 使用 RAM read data、Linefill Buffer 和 BIU beat 决定返回 data、发起 miss request 或 stall。
first fetch	PFU 标记的“一个新的顺序取指区间中的第一笔 fetch”。它通常出现在新 cache line、force 或其他非顺序取指之后, 要求 ICU 重新执行完整 tag/data lookup。first 不是 BIU linefill 的第一个 beat, 不表示当前地址必然 cache miss, 也不只表示 reset 后的第一条指令。
reduced lookup	顺序 fetch 已知上一笔所在 way 时, 只读必要 data bank 或不读 tag 的省电 lookup。
Linefill Buffer (LFB)	ICU 内部 4-entry doubleword buffer, 保存 BIU 返回的 cache line 或 non-cacheable burst beat, 并可直接 forward 给 PFU。
critical doubleword	当前 PFU 正在等待的那个 64 位双字。cacheable linefill 从该双字开始返回, 因此无需等完整 cache line 到齐即可恢复当前取指。

Term	Meaning
forward / 前递	数据不等待写入I-cache RAM，直接从BIU当前返回拍或LFB已有效槽送给PFU。前递不等于cache hit。
corkscrew	I-cache data RAM的交叉编排方式。偶数doubleword中way0位于bank0，奇数doubleword中way0位于bank1；way1使用另一bank。
pseudo-random replacement	准随机替换。当前RTL用一个随PFU请求翻转的全局位选择默认way；它不是Least Recently Used，也不是每个set一个替换状态机。
Modified Virtual Address (MVA)	ARM cache maintenance 术语，表示按虚拟地址指定要 invalidate 的 cache line。
invalidate by MVA	cache maintenance 操作之一，查找指定 MVA 对应的 tag，如果命中则清除命中 way 的 valid bit，并同步处理 LFB。
invalidate all	cache maintenance 操作之一，后台遍历所有 tag index，清除两个 way 的 valid bit。
Error Correction Code (ECC)	tag/data RAM 上的错误检测/纠正编码。ICU 检测 syndrome、记录 error location，并向 PFU/system 报告。
I-cache Error Bank Register (IEBR)	ICU 内部两个 error bank register，用于记录 ECC error 位置、mask 后续相同位置 error，并阻止 linefill 分配到对应 way/index。
Error Bank Register (EBR)	PPB 可读写的 error bank register 接口名；在 ICU 内部对应 <code>iebr0</code> / <code>iebr1</code> 。
cache maintenance (CM)	cache 维护操作的统称；本文主要包括 invalidate by MVA 和 invalidate all。RTL 端口名中常缩写为 <code>cm</code> 。
invalidate all (IA)	cache maintenance 操作之一，表示清除全部 I-cache tag valid bit。Timing 图中用 <code>IA</code> 作短标签。
ICERR / ICDET	ICU 对外的 I-cache error report/detail bus。TBD：当前 ICU 源码没有展开这两个信号名的正式全称；本文按外部错误记录和错误类型详情解释。
fake hit	ICU 在已有 ECC error 正在处理时，为避免使用不可信 RAM data，给 PFU 返回一个需要 replay 的假 data-valid 事件。
cancel	PFU 表示当前 outstanding fetch response 不再需要。ICU 清除 Icache fetch；如果已有 BIU single transaction 在途，后续 data 会被 mask 到 transaction last。
replay	PFU 因 ECC 或 fake hit 等原因丢弃当前 data 并重新取同一 fetch 的行为。ICU 只负责输出 error/replay 条件，不执行 PFU FIFO replay。
Reset All Registers (RAR)	RTL 配置参数；为 1 时更多寄存器受 <code>reset_n</code> / <code>po_reset_n</code> 复位，为 0 时部分非关键寄存器不复位。

1.1 模块目的与职责

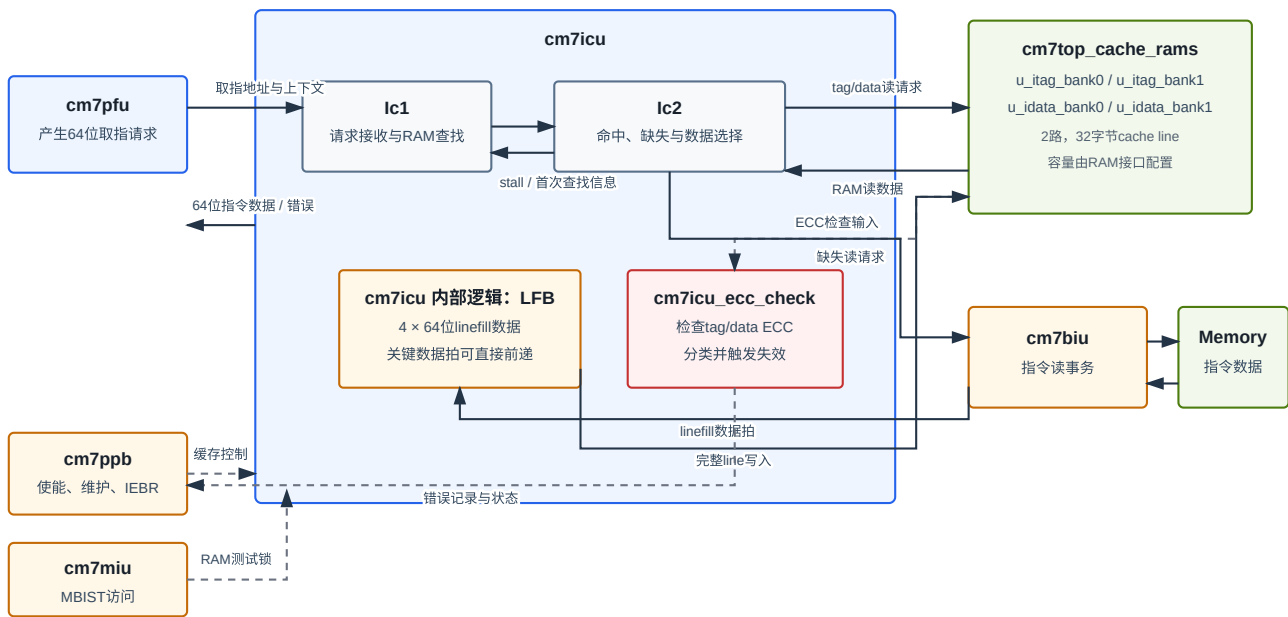
ICU 是 PFU 和 instruction memory system 之间的 64-bit instruction data cache controller。它必须复现以下核心职责：

- 对 PFU 的 fetch request 执行 `ack / dvalid` 协议，保持每次返回 64-bit instruction data。
- 在 I-cache enable 且 memory attribute cacheable 时访问 2-way I-cache，命中时直接返回 cache data。
- 未命中时向 BIU 发起 linefill 或 single read，把 BIU beat 写入 LFB，并尽早把 critical doubleword forward 给 PFU。
- 对完整 cacheable linefill 执行 cache allocation；对 non-cacheable 或不可分配 fetch 只使用 LFB/BIU forward，不写入有效 cache line。
- 执行 cache maintenance，包括 invalidate by MVA、invalidate all、Instruction Synchronization Barrier (ISB) 对 LFB 的影响，以及与 PFU fetch 的 stall/priority。
- 检测 tag/data RAM ECC error，更新 IEBR，发起 ECC invalidate，向 PFU 和 system error 端口报告。
- 支持 MIU/MBIST 对 I-cache RAM 的直接读写，并在 MBIST 与正常 fetch/ECC/maintenance 之间仲裁 RAM 访问。

1.2 宏观指令数据流

图例：Random Access Memory (RAM，随机存取存储器)、Memory Built-In Self-Test (MBIST，存储器内建自测试)。图中使用实际RTL模块名；LFB和Ic1/Ic2不是独立module，因此明确标为 `cm7icu` 内部逻辑。

cm7icu 宏观架构与指令数据流



读图先看上半部的命中路径。 `cm7pfu` 把双字对齐取指地址、first标记、权限和向量取指导引送入 `cm7icu`。 `Ic1` 决定是否读取tag/data RAM， `Ic2` 在下一拍接收RAM结果，完成tag比较、data bank选择和ECC检查，然后把64位指令数据返回 `cm7pfu`。地址从左向右流动，RAM读数据与最终指令数据从右向左返回。

再看下半部的缺失路径。Ic2没有在cache、LFB或当前BIU返回拍中找到目标数据时，向 `cm7biu` 发出读请求。外部 memory的数据拍先回到ICU：当前PFU等待的数据拍可直接送PFU，其余数据进入LFB。cacheable line的4个64位双字全部收齐后，LFB再通过allocation路径把完整line写入I-cache RAM。这个设计同时降低首次缺失延迟并保留后续局部命中能力。

图中的I-cache RAM位于 `cm7icu` 端口之外。当前通用集成模型的实际module是 `cm7top_cache_rams`，其中有两个 tag bank和两个data bank；芯片实现可以替换成等价RAM macro。`cm7icu_ecc_check` 是 `cm7icu` 内部实际实例，读取同一批tag/data结果并产生syndrome分类。`cm7ppb` 和 `cm7miu` 不传输普通指令payload，但分别控制cache使能/维护/错误记录和MBIST独占访问，因此会改变主数据流能否查cache、何时失效以及谁能使用RAM端口。

本章只保留宏观 instruction data flow。Ic1/Ic2 的逐 cycle 行为、LFB/cache RAM 结构、ECC 和 cache maintenance 时序放在后续章节对应功能下展开。

2. 模块组成

ICU 这个大模块由两个实际 RTL module、若干 `cm7icu` 内部功能块，以及多组接口 assertion spec 共同定义行为边界。

Module / Block	Actual Name	Function
ICU top-level	<code>cm7icu</code>	集成 PFU fetch pipeline、I-cache RAM arbitration、hit/miss selection、LFB、BIU request、cache maintenance、IEBR、MBIST mux 和所有外部输出。
ECC checker	<code>cm7icu_ecc_check</code>	对 tag0/tag1/data0/data1 read data 计算 ECC syndrome，输出 error valid、fatal/correctable、tag/data bank、raw way 和 ICDET 分类。
Ic1 stage logic	<code>cm7icu</code> 内部逻辑	接收 PFU request，产生 lookup request、tag/data RAM enable/address，并处理 first/reduced lookup 策略。
Ic2 stage logic	<code>cm7icu</code> 内部逻辑	保存 accepted fetch context，判断 cache hit、LFB hit、BIU hit、miss、fake hit，并生成 PFU response 和 BIU miss request。
Linefill Buffer	<code>cm7icu</code> 内部逻辑	保存一个 cache line 的 4 个 64-bit beat、valid mask、allocation way、allocated qword mask、outstanding transaction mask。
Cache allocation control	<code>cm7icu</code> 内部逻辑	根据 LFB valid mask、cacheability、IEBR block、ECC block 和 pseudo-random/default way 选择 replacement way，分两次写入 data RAM/tag RAM。
Cache maintenance control	<code>cm7icu</code> 内部逻辑	实现 invalidate by MVA Finite State Machine (FSM) 和 invalidate all background walker，并输出 <code>icu_lsu_cm_in_prog_o</code> 。
Error bank registers	<code>cm7icu</code> 内部逻辑	实现 <code>iebr0</code> / <code>iebr1</code> 字段、PPB read/write、ECC auto allocation、error masking、external ICERR report。
MBIST path	<code>cm7icu</code> 内部逻辑	处理 MIU lock/ack、MBISTALL、tag/data bank select、read latency、write data routing和 conflict error。
Interface specs	<code>cm7_pfu_icu</code> 、 <code>cm7_icu_biu</code> 、 <code>cm7_icu_ram</code> 、 <code>cm7_dpu_icu</code> 、 <code>cm7_miu_icu</code> 、 <code>cm7_ppb_icu</code>	这些文件主要是 assertion/interface contract，不是功能实现；它们定义 request/ack/data-valid、RAM latency、MBIST four-state handshake、cache maintenance commit 语义等外部约束。

3. I-cache架构与行为

3.0 I-cache概念基础

本节先解释后文反复出现的cache概念。这里讲的是 `cm7icu` 管理的Instruction cache (I-cache，指令缓存)，不是data cache。I-cache只服务instruction fetch：PFU给出取指地址，ICU先尝试从片上I-cache RAM找到数据；找不到时才通过BIU读取下一级instruction memory。

程序执行通常具有空间局部性：当前指令附近的后续指令很快也会执行。I-cache因此以32字节cache line保存数据，而不是只保存一次请求的64位。一次外部读取可以让同一line中的后续四个双字都在本地命中。I-cache也利用时间局部性：循环或重复调用再次访问同一line时，只要该line未被替换或失效，就不需要再访问外部memory。

本实现不接收store、不维护dirty bit，也不执行data cache意义上的write-back或write-through。memory attribute中的外部缓存策略仍会传给BIU，但 `cm7icu` 自身只使用cacheable位决定“能否查I-cache以及能否把linefill分配进去”。ICU对I-cache RAM的普通写入只有linefill allocation和tag invalidation两类。

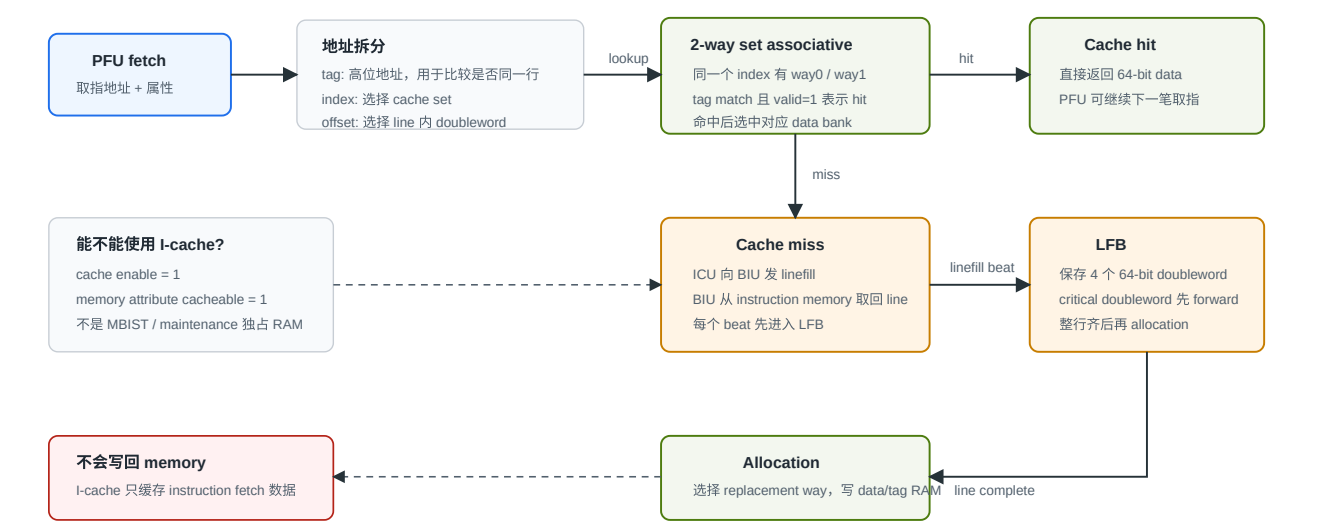
通用 cache 可以按“局部性、cache line、地址拆分、命中判断、miss 后填充”这条链来理解。CPU 取指通常有空间局部性：当前指令附近的后续指令很可能马上被取到，所以 cache 不只保存当前 64-bit doubleword，而是一次保存一整条 cache line。之后同一 line 内的 fetch 可以直接从 I-cache 返回，不需要再次访问外部 instruction memory。

通用 cache 概念	一般含义	在 <code>cm7icu</code> 中的映射
cache line	cache 和 memory 之间搬运、保存、失效的基本块。line 比单次 CPU 请求更大，用来利用顺序访问的空间局部性。	1 条 I-cache line 是 32 bytes，即 4 个 64-bit doubleword。BIU cacheable linefill 预期返回 4 个 beat。
offset	地址低位，选择 cache line 内的具体 byte/word/doubleword。	<code>addr[4:3]</code> 选择 4 个 64-bit doubleword 之一；更低位由 PFU/取指对齐逻辑在 64-bit data 内解释。
index	地址中间位，选择 cache 中要查找的 set。	<code>addr[14:5]</code> 选择 I-cache set，并作为 tag/data RAM address 的主要字段。
tag	地址高位，用于区分映射到同一 index 的不同 memory line。	<code>addr[31:11]</code> 与 tag RAM 中保存的 tag 比较；只有 valid 且 tag 相等才是 cache hit。
set associative	每个 index 不是只有一个位置，而是有多个 way，可减少两个热点地址互相替换的问题。	I-cache 是 2-way set-associative。每个 set 有 way0 和 way1；lookup 时两个 way 的 tag 并行比较。
valid bit	标记某个 cache line 是否包含可用内容。valid 为 0 时即使 data RAM 里有旧数据也不能命中。	tag RAM entry 中带 valid bit。invalidate by MVA 或 invalidate all 主要清 valid，不需要清 data RAM。
cache hit	index 选中的 set 中，有至少一个 way 的 valid bit 为 1 且 tag match。	Ic2 stage 用 RAM read data 做 tag compare；命中后从对应 data bank 选 64-bit data 返回 PFU。
cache miss	查不到有效 tag match，需要访问下一级 memory。	Ic2 miss 后向 BIU 发 linefill 或 single read；返回 beat 先进入 LFB，再可能 allocation 到 I-cache RAM。
allocation	miss 后把取回的 line 放入 cache 的过程。若 set 内多个 way 可用，需要选择 replacement way。	LFB 四个 doubleword 全部 valid 后，ICU 分 lower/upper 两次写 data/tag RAM，并在第二次 tag write 时置 valid。

通用 cache 概念	一般含义	在 cm7icu 中的映射
maintenance	软件或系统事件显式改变 cache 可用内容，例如 invalidate。	PPB/DPU 发起 invalidate by MVA、invalidate all，ICU 清 tag valid，并同步处理匹配的 LFB。

把上表转换成一次实际 fetch：PFU 给出地址后，ICU 先用 index 找到一个 set，同时读取 way0/way1 的 tag 和 data；再用 tag 判断哪个 way 有效；最后用 offset 从命中的 line 中选出本次需要的 64-bit doubleword。若没有 hit，ICU 不能随便从 data RAM 取旧值，而必须通过 BIU 重新取 instruction memory，并按 cacheable 属性决定是否把取回的 line 变成新的 valid cache line。

cm7icu I-cache 基础数据流



读这张图时先看上方 hit path：PFU 发来 fetch address 和属性，ICU 把地址拆成 tag、index 和 offset。index 选择 I-cache 的一个 set；每个 set 有 way0 和 way1 两个候选 cache line。ICU 读取两个 way 的 tag/data RAM，如果某个 way 的 valid bit 为 1 且 tag 等于当前地址高位，就发生 cache hit；这时 ICU 直接从对应 data bank 选出 64-bit doubleword 返回 PFU。cache hit 数据不会先写入或经过 LFB，LFB 不是 I-cache 与 PFU 之间的流水寄存器。

如果两个 way 都没有命中，就发生 cache miss。miss 时 ICU 通过 BIU 请求 instruction memory。BIU 当前返回的 beat 若正好是当前 fetch 等待的 critical doubleword，可以在同一拍直接 forward 给 PFU；对于普通 linefill，该 beat 同时写入 LFB 对应槽。较早拍已经写入 LFB 的数据，也可以在后续 fetch 时从 LFB forward 给 PFU。因此数据不需要等待整条 line 写进 cache 后才返回，但也不能理解为“所有返回数据都必须先经过 LFB”。等同一条 cache line 的 4 个 64-bit doubleword 都有效后，如果这次 fetch 允许 cache allocation，ICU 再把整条 line 从 LFB 写进 I-cache RAM。

是否允许使用和写入 I-cache 由两个层次共同决定。第一层是配置：dpu_icu_ccr_icen_i 表示 I-cache 是否 enable。第二层是 memory attribute：pfu_icu_attrs_i 来自 MPU/PFU 侧，其中 cacheable 属性决定该地址是否可以缓存。只有 cache enable、attribute cacheable、lookup 没有被 MBIST/maintenance 阻止、且目标 way 没有被 IEBR/ECC block 时，miss 后的 linefill 才会变成有效 I-cache allocation。否则 ICU 仍然可以通过 BIU/LFB 把数据返回给 PFU，但不会把它作为有效 cache line 保存下来。

对重写实现来说，cache 相关行为可以分成四个独立问题：

问题	本文对应章节	必须复现的行为
这次fetch要不要查cache?	3.3 查找策略	根据first/small/reduced/no-lookup规则决定tag/data RAM访问。
命中、LFB、BIU同时可能给数据时选谁?	3.4 数据来源优先级	当前BIU拍优先，其次LFB，再次I-cache RAM。
miss后怎么取回数据?	3.5 LFB与BIU	linefill/single request、valid mask、cancel和masked transaction。
取回的line写到哪个way?	3.6 RAM组织与分配	replacement way、IEBR/ECC阻塞和tag valid写入顺序。

几个容易混淆的点：

- `hit` 不是“地址在最近访问过”，而是“当前 index 的某个 way 有 valid tag match”。
- `miss` 不一定最后写入 I-cache；non-cacheable、cache disabled、way 被 block 或 linefill fault 都可能让这次 fetch 只通过 BIU/LFB 返回。
- I-cache hit数据从data RAM直接返回PFU，不会先进入LFB；LFB保存BIU取回的普通linefill或非-cacheable burst数据，其中只有满足allocation条件的完整cache line随后会写入I-cache。
- `LFB forward` 不等于 cache hit。它只是 miss linefill 过程中，从 LFB 临时转发已经回来的 doubleword。
- LFB向PFU返回只要求当前目标doubleword有效，不要求4个槽全部有效；只有向I-cache执行allocation才要求valid mask为 1111。
- `allocation` 不是 PFU 发 request 时发生，而是 LFB 收齐完整 cache line 后，ICU 再分两次把 data/tag RAM 写完。
- `invalidate` 通常清除 tag valid bit，不需要把 data RAM 内容真正清零；只要 valid=0，后续 lookup 就不会把旧 data 当作 hit。
- `replacement way` 只在允许 allocation 时有意义；如果两个 way 都被 IEBR/ECC block，则不产生有效 allocation。

3.1 Cache容量、地址拆分与物理组织

I-cache为2-way set-associative（2路组相联），cache line固定为32字节。容量变化通过set数量实现，way数和line大小不变。

3.1.1 什么是“2路组相联”

cache的关联度描述“一个memory cache line映射到某个set后，在这个set中有多少个候选存放位置”。常见组织方式如下：

组织方式	每个地址的候选位置	命中查找	主要优点	主要代价
direct-mapped（直接映射）	所选set中只有1个way	比较1个tag	RAM和比较器最少，访问简单	映射到同一set的两个热点line会不断互相替换。

组织方式	每个地址的 候选位置	命中查找	主要优点	主要代价
N-way set-associative (N路组相联)	所选set中有 N个way	N个tag并 行比较	在硬件成本和冲 突miss之间折中	需要N个tag候选、命中选择 和miss替换策略。
fully associative (全相 联)	cache中任意 line位置	与全部有 效tag比较	几乎没有固定 index造成的冲突	比较器和选择网络随总line 数增长，不适合本ICU规 模。

本ICU采用2-way set-associative：地址先通过index选择唯一的set，随后只在这个set的way0和way1中查找。它不会搜索其他set。与direct-mapped相比，同一个index可以同时保存两个不同tag对应的memory line，因此两个冲突热点可以共存；与fully associative相比，每次只比较两个tag，硬件和功耗可控。

这里必须区分“整个cache有多少个set”和“一次lookup访问多少个set”：

- 整个I-cache有很多个set。根据容量不同，共有64、128、256、512或1024个set。
- 一次fetch只用index从这些set中选中的一个，因此一次lookup只访问一个set。
- 被选中的这个set内部有way0和way1两个候选line，所以需要并行比较两个tag。
- 图中的 `set k` 表示“本次地址选中的第k个set”，不是说整个I-cache只有一个set。

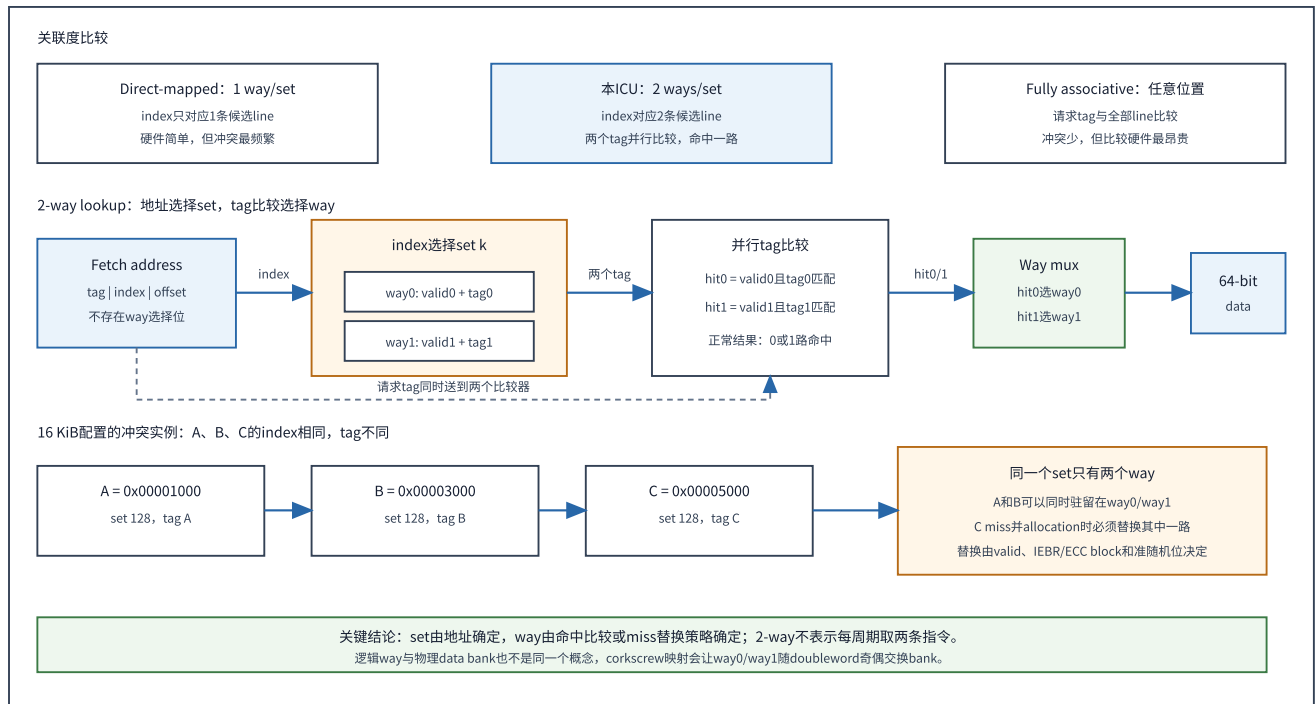
以16 KiB配置为例，整个I-cache有256个set，每个set有2个way，每个way位置保存一条32-byte cache line：

```
整个I-cache: set 0, set 1, ... , set 255

一次fetch:
  addr[12:5] = 128
    |
    +--> 只选择set 128
           |- way0: valid0 + tag0 + 32-byte data
           `- way1: valid1 + tag1 + 32-byte data

总容量 = 256 sets × 2 ways/set × 32 bytes/way = 16 KiB
```

因此index仍然不可缺少。如果没有index，ICU就无法从256个set中确定本次应该读取哪一组tag/data；如果每次把256个set全部读取并比较，就会变成接近fully associative的昂贵结构，而不是当前2-way set-associative实现。



图的上半部分先比较三种关联方式：direct-mapped每个set只有一个候选位置，本ICU每个set有两个候选way，fully associative则允许任意位置。中间是本ICU真正的lookup路径：请求地址的index从全部set中选择一个set k，同时请求tag送到这个set内部的way0和way1比较器。valid0/tag0 匹配产生way0 hit，valid1/tag1 匹配产生way1 hit；命中mux据此选择对应的64-bit data。地址中没有way选择位，因为在完成tag比较之前并不知道目标line位于哪一路。

图的下半部分使用16 KiB配置说明冲突。此时有256个set，0x00001000、0x00003000和0x00005000的addr[12:5]都等于set 128，但高位tag不同。A和B可以分别驻留在way0和way1；C到来时两个way都已占用，C会miss，并在允许allocation时替换其中一路。2-way减少了A/B之间的冲突，但无法同时容纳第三个同set line，所以并没有消除所有conflict miss（冲突缺失）。

3.1.2 地址怎样映射到set

把byte address先按32字节line对齐，可以得到memory line number：

```
line_number = address / 32
set_index   = line_number mod set_count
tag         = line_number / set_count
```

硬件不需要真正执行除法和取模，因为line大小和set数量都是2的幂。addr[4:0]自然成为line offset；紧随其后的若干位成为index；剩余高位成为tag。例如16 KiB配置有256个set，使用addr[12:5]作为8-bit index。这个8-bit值可以表示0到255，因此每个地址只选择256个set中的一个；其余有效高位用于区分映射到同一set的不同line。

这种映射意味着：相差set_count × line_size字节的地址会映射到同一个set。16 KiB配置中该间隔为256 × 32 = 8192字节，即0x2000，所以前述A、B、C都选择set 128。容量改变后set数量和冲突间隔也随之改变，但每个set仍固定只有两个way。

3.1.3 命中时怎样选择way

full lookup在Ic1读取所选set的两路tag和两个data bank；Ic2并行计算两个逻辑命中：

1. 对应way的tag entry必须valid。
2. 请求memory attribute必须允许使用I-cache。
3. 请求tag必须与该way保存的有效tag位相等。
4. 该way/index不能被IEBR或当前ECC错误屏蔽。
5. RAM读结果必须属于当前有效lookup，不能被fake hit、maintenance或MBIST上下文冒用。

正常情况下结果只允许三种：way0 hit、way1 hit或两路都miss。两路同时hit意味着同一个set中出现了重复有效tag，属于不应发生的状态；RTL assertion要求在没有ECC/fake-hit干扰时不得出现这种情况。命中哪个way只决定从哪条逻辑cache line取数据，不改变程序地址。

3.1.4 miss时怎样选择replacement way

两路都不命中时，地址仍然只允许写入当前index选择的set，不能为了避免替换而放到其他set。replacement way按以下顺序决定：

1. 若某一路被IEBR或正在处理的ECC错误block，优先选择另一条未block的way。
2. 若两路都被block，本次linefill仍可通过BIU/LFB返回指令，但不能形成有效cache allocation。
3. 两路均未block且只有一路valid时，选择invalid way，保留已有的有效line。
4. 两路均valid或均invalid时，使用随PFU请求翻转的全局 `default_alloc_way0` 位做准随机选择。

本ICU不是Least Recently Used (LRU，最近最少使用)策略。它没有为每个set保存“最近访问的是way0还是way1”的替换状态，也不是每个entry各有一个小状态机。

`default_alloc_way0` 在整个ICU中只有一个1-bit寄存器，由所有set共同使用。它不是“每个set一位”，也不属于某个way、cache line或LFB entry。该位的含义是：当本次替换已经排除block约束，并且所选set的两个way具有相同valid状态时，值为1选择way0，值为0选择way1。只要两个way中恰好有一个invalid，就直接选择invalid way，不读取该默认位；只要某一路被block，就优先选择未block的另一路。

这个全局位在复位时为0，并在每个PFU fetch request出现时翻转，包括最终cache hit的请求，而不是只在cache miss或allocation时翻转。因此它表达的是全局取指请求历史，不是某个set的使用历史。举例来说，假设访问set 12发生miss且两路都valid，在replacement判定时全局位为1，则选择set 12的way0；之后对其他set的PFU请求也会继续翻转该位。下一次再访问set 12并发生相同类型的miss时，选择结果可能已被这些无关set的请求改变。由此可见，该机制只提供低成本的近似交替，不能推导“当前set中哪一路最久没有使用”。

重实现时应把它建模成ICU级共享状态，而不是按set建立位数组。等价的选择逻辑可写为：

```
if exactly_one_way_is_blocked:
    replacement_way = the_unblocked_way
else if both_ways_are_blocked:
    allocation = disabled
else if way0.valid != way1.valid:
    replacement_way = the_invalid_way
else:
    replacement_way = default_alloc_way0 ? way0 : way1
```

3.1.5 way、bank和双发射不是同一个概念

- **way**是同一set内可以保存不同tag line的逻辑候选位置。
- **data bank**是I-cache RAM的物理访问分组。由于corkscrew布局，way0的偶数doubleword在bank0、奇数doubleword在bank1；way1相反。因此不能把way0永久等同于bank0。
- **2-way**不表示ICU每cycle取两条指令，也不表示PFU的两个instruction slot。ICU接口每次返回一个64-bit doubleword，后续由PFU从其中组织Thumb指令。
- **两个way并行读取**是为了在tag比较完成后立即选择命中数据；它不代表软件同时访问两个地址。

对重实现来说，必须保持“地址只选set、两个tag并行比较、命中选择way、miss在同set内选择replacement way”这四步。物理RAM可以换成不同macro，但不能把way错误地编码成地址中的固定选择位，也不能把准随机替换改成每set LRU后仍声称cycle-accurate等价。

容量编码	容量	set数量	有效index地址位	每路line数	总数据容量
0000	4 KiB	64	addr[10:5]	64	2 × 64 × 32 B
0001	8 KiB	128	addr[11:5]	128	2 × 128 × 32 B
0011	16 KiB	256	addr[12:5]	256	2 × 256 × 32 B
0111	32 KiB	512	addr[13:5]	512	2 × 512 × 32 B
1111	64 KiB	1024	addr[14:5]	1024	2 × 1024 × 32 B

ram_icu_cache_size_i 只能使用上述五种编码，并且复位后不得改变。当前仓库的通用 cm7top_cache_rams 模型把I-cache配置为16 KiB；cm7icu 本身必须支持全部五种容量。物理接口始终提供10位tag RAM地址和12位data RAM地址，小容量通过mask忽略高位。

I-cache的set、way与cache line结构

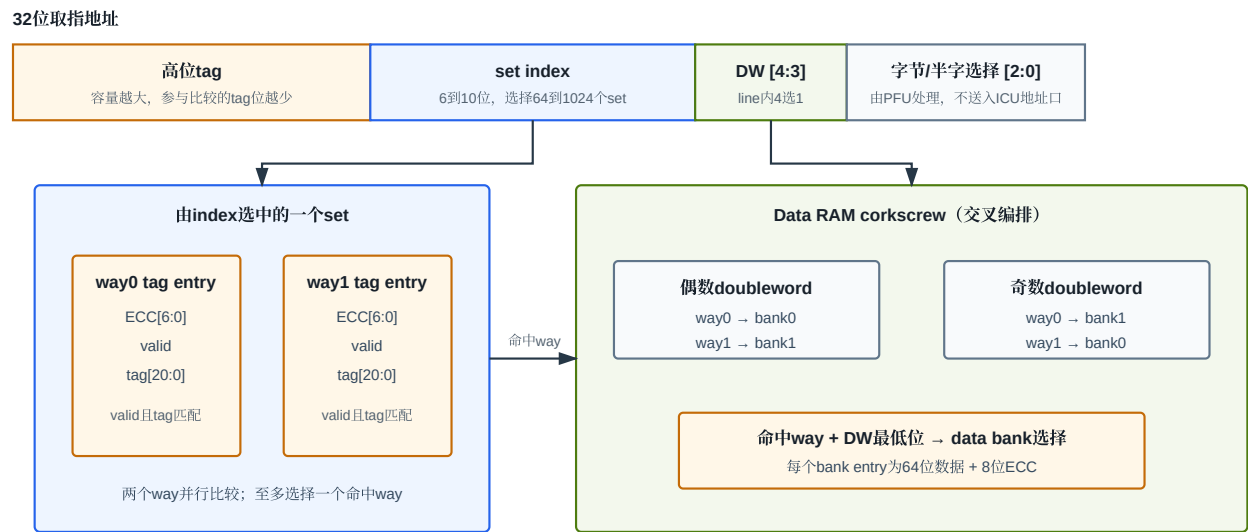


这张图专门说明set和way的层次关系。整块I-cache沿纵向分成N个set，N随容量从64变化到1024；每个set横向固定包含way0和way1两个位置。每个way位置保存一条32字节cache line，对应4个64位doubleword，并配有自己的valid、tag和ECC。因此“2-way”不是整块cache只有两条line，而是每一个set都能同时保存两个映射到相同index、但tag不同的cache line。

取指地址的index只选择一个set，例如图中的set k；ICU不会在其他set中搜索。地址中不存在“way选择位”：set k的way0 tag和way1 tag会并行与请求tag比较，哪一路valid且tag匹配，命中结果就选择哪一路的数据；两路都不匹配就是cache miss。miss后的way也不是由地址决定，而是按“IEBR/ECC阻塞、valid状态、准随机默认位”的替换策略在set k的两个way之间选择。容量公式为 $\text{set数量} \times 2 \text{ ways} \times 32 \text{ bytes}$ ，例如16 KiB配置有256个set，共512条cache line。

各容量使用的set index位如下：4 KiB使用 $\text{addr}[10:5]$ ，8 KiB使用 $\text{addr}[11:5]$ ，16 KiB使用 $\text{addr}[12:5]$ ，32 KiB使用 $\text{addr}[13:5]$ ，64 KiB使用 $\text{addr}[14:5]$ 。 $\text{addr}[4:3]$ 只选择32字节cache line中的4个64位doubleword之一；其中 $\text{addr}[3]$ 还参与corkscrew data-bank选择，但它仍然不是way选择位。剩余有效高位作为tag，用于判断所选set中的way0或way1是否保存了当前地址对应的line。

I-cache地址拆分、set与bank组织



容量变化只改变有效index位数和tag比较掩码；物理端口仍保持10位tag地址、12位data地址。

读图先从顶部地址条开始。 $\text{addr}[4:0]$ 是32字节line内偏移，其中 $\text{addr}[4:3]$ 选择四个64位doubleword； $\text{addr}[2:0]$ 选择双字内部字节，PFU向ICU只发送 $[31:3]$ ，所以这三位由PFU后续选半字/指令时处理。中间index选择一个set，剩余高位作为tag。容量越大，index多占一位，tag比较就少一位；RTL通过 `index_mask` 和 `tag_mask` 实现这个可变边界。

图左下说明tag查找。所选set的way0和way1并行读出，每项格式为 $\{\text{ECC}[6:0], \text{valid}, \text{tag}[20:0]\}$ 。命中必须同时满足：本次确实有效lookup、该way的valid为1、未被IEBR/ECC屏蔽、有效tag位与请求地址相同，而且memory attribute表示cacheable。RAM中的旧data即使数值碰巧正确，只要valid为0就绝不能命中。

图右下说明corkscrew交叉编排。data RAM有bank0和bank1，每项为 $\{\text{ECC}[7:0], \text{data}[63:0]\}$ 。偶数doubleword中way0位于bank0、way1位于bank1；奇数doubleword交换。Ic2用“命中way”和 $\text{addr}[3]$ 共同选择bank。这样两个way的数据能在一次双bank读取中同时候选，又能在allocation时每拍把相邻两个doubleword并行写入两个bank。

3.1.6 corkscrew data-bank布局

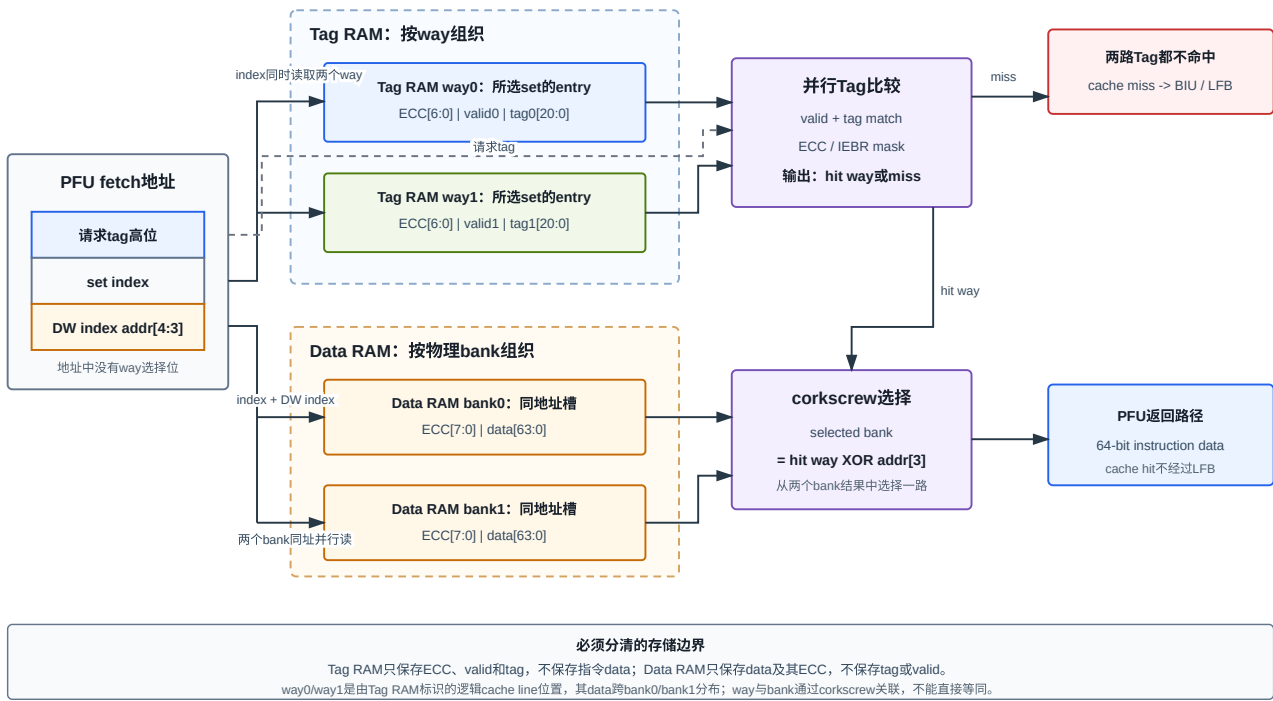
corkscrew 是本ICU对I-cache data RAM采用的交叉存放布局，不是cache替换策略，也不是一种新的way组织方式。每个set仍然只有way0和way1两个逻辑候选位置； bank0 和 bank1 是承载这两个way数据的两个物理RAM分组。所谓交叉，是指同一个way的一条32字节cache line不会全部固定放在某一个bank中，而是按64-bit doubleword编号的奇偶在两个bank之间交替存放。

在展开corkscrew之前，必须先区分I-cache中的两类独立存储：

- **Tag RAM**回答“当前地址是否已经缓存在这个set中，以及命中哪个way”。它按way0和way1组织，每个entry保存tag ECC、 valid和tag，不保存指令数据。
- **Data RAM**回答“命中后应该返回哪一个64-bit instruction doubleword”。它按物理bank0和bank1组织，每个entry保存data ECC和64-bit data，不保存tag或valid。

cm7icu I-cache的Tag RAM与Data RAM分离结构

Tag RAM保存“该地址是否命中”，Data RAM保存“命中后返回的64-bit指令数据”；两者是独立RAM路径。



这张图先从完整lookup路径说明两类RAM的边界。PFU fetch地址中的set index同时读取Tag RAM way0和way1的entry，请求地址高位再与两个entry中的有效tag并行比较。比较结果只能是way0 hit、way1 hit或miss；这里产生的 hit way 是后续data选择的控制信息，不是地址中的固定way位。

Data RAM走独立路径。{set index, DW index} 作为相同的读地址并行送到bank0和bank1，每个bank返回一个64-bit候选数据。Ic2取得Tag RAM产生的 hit way 后，再结合 addr[3] 执行corkscrew bank选择，把正确的一个doubleword送到PFU。也就是说，数据路径是“Tag RAM决定命中way，Data RAM提供两个bank候选，选择器把二者关联起来”，不是从data bank中读取tag。

Tag RAM entry与Data RAM entry的内容必须严格分开：

RAM类型	组织方式	每个entry保存	不保存
Tag RAM	way0、way1各一组；set index寻址	ECC[6:0] + valid + tag[20:0]	instruction data

RAM类型	组织方式	每个entry保存	不保存
Data RAM	bank0、bank1各一组；{set_index, DW_index} 寻址	ECC[7:0] + data[63:0]	tag、tag valid

一个Tag RAM valid bit管理对应way中的整条32字节cache line。只有tag匹配且valid=1时，该way分散在Data RAM中的4个doubleword才允许作为cache hit数据使用；Data RAM中的旧数值即使仍然存在，只要Tag RAM valid=0就不能命中。linefill allocation时，ICU分别写Data RAM与目标way的Tag RAM：先写前两个data doubleword并保持tag valid=0，再写后两个data doubleword并把tag valid置1。因此tag和data在逻辑上属于同一条cache line，但物理上位于不同RAM中。

下面的表和corkscrew图只描述Data RAM内部布局，不包含Tag RAM。一条32字节cache line包含4个64-bit doubleword。addr[4:3] 给出line内doubleword编号，addr[3] 是该编号的最低位，因此它区分偶数doubleword和奇数doubleword。具体映射如下：

Line内数据	地址偏移	addr[4:3]	bank0同地址槽保存	bank1同地址槽保存
DW0	+0x00	00	way0.DW0	way1.DW0
DW1	+0x08	01	way1.DW1	way0.DW1
DW2	+0x10	10	way0.DW2	way1.DW2
DW3	+0x18	11	way1.DW3	way0.DW3

若把way0编号为0、way1编号为1，把bank0编号为0、bank1编号为1，则逻辑way到物理bank的选择关系可以写成：

```
selected_bank = hit_way_number XOR addr[3]
```

这条公式的设计含义是：way0的偶数doubleword在bank0、奇数doubleword在bank1；way1正好相反。addr[3] 本身不选择way，way仍然由两个tag的valid和比较结果决定。只有在知道命中way以后，Ic2才能把“命中way”和addr[3] 组合起来，从两个bank的读结果中选择正确数据。

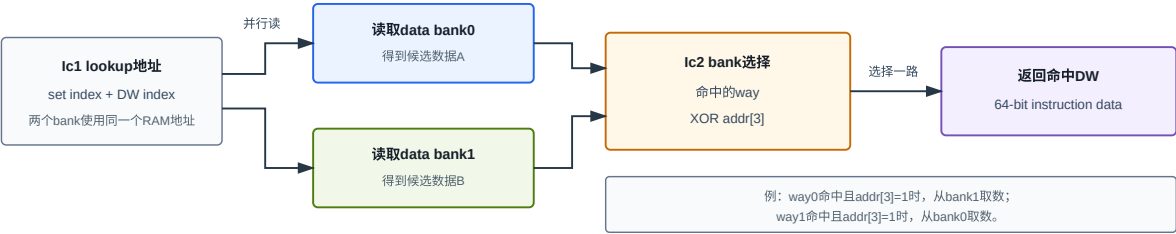
cm7icu Data RAM内部的corkscrew交叉布局

本图只展开Data RAM；Tag RAM及其valid/tag字段是独立结构。一条32字节line包含4个64-bit doubleword。

1. 所选set中两个way的数据如何分布

行内位置	addr[4:3]	data bank0 同地址槽	data bank1 同地址槽	映射规则 way0：偶数DW在bank0，奇数DW在bank1 way1：偶数DW在bank1，奇数DW在bank0 bank = way_number XOR addr[3] way0编号为0，way1编号为1 addr[3]是64-bit doubleword编号的奇偶位
DW0 / +0x00	00	way0.DW0	way1.DW0	
DW1 / +0x08	01	way1.DW1	way0.DW1	
DW2 / +0x10	10	way0.DW2	way1.DW2	
DW3 / +0x18	11	way1.DW3	way0.DW3	

2. Cache hit时的读取路径



3. Linefill完成后的两拍写入

目标way	allocation拍	bank0写入	bank1写入	结果
way0	第1拍	DW0	DW1	每拍两个bank并行写 两拍写完DW0至DW3
	第2拍	DW2	DW3	
way1	第1拍	DW1	DW0	同样两拍写完 bank分配与way0相反
	第2拍	DW3	DW2	

图的顶部是物理存放矩阵。蓝色单元表示way0的数据，绿色单元表示way1的数据。对同一个set和同一个doubleword地址，bank0与bank1各保存一个way的候选数据；当doubleword编号从偶数变为奇数时，两个way所在的bank互换。这里的“同地址槽”是指两个data RAM收到相同的 {set index, doubleword index} 读地址，并不表示两个bank保存相同数据。

图的中部是cache hit读取路径。Ic1把同一个data RAM地址送给bank0和bank1，两路RAM并行产生候选数据。与此同时，tag RAM判断当前请求命中way0还是way1。进入Ic2后，选择逻辑使用命中way和 addr[3] 确定实际bank，再把该bank的64-bit doubleword送入后续PFU返回数据路径。例如，way0命中且请求DW1时，数据来自bank1；way1命中且同样请求DW1时，数据来自bank0。顺序取指已经记住命中way时，reduced lookup可以提前只使能实际需要一个bank，从而减少不必要的RAM翻转功耗。

图的底部是linefill完成后的allocation写入路径。由于一条way的相邻doubleword被分散到不同bank，两个bank可以在同一拍各写一个doubleword。目标为way0时，第一拍并行写bank0的DW0和bank1的DW1，第二拍并行写bank0的DW2和bank1的DW3；目标为way1时bank关系相反，第一拍写bank0的DW1和bank1的DW0，第二拍写bank0的DW3和bank1的DW2。因此4个doubleword的数据只需两拍写完。tag在第一拍以valid=0写入，在第二拍数据写完时才以valid=1写入，防止未完整写入的cache line被lookup命中。

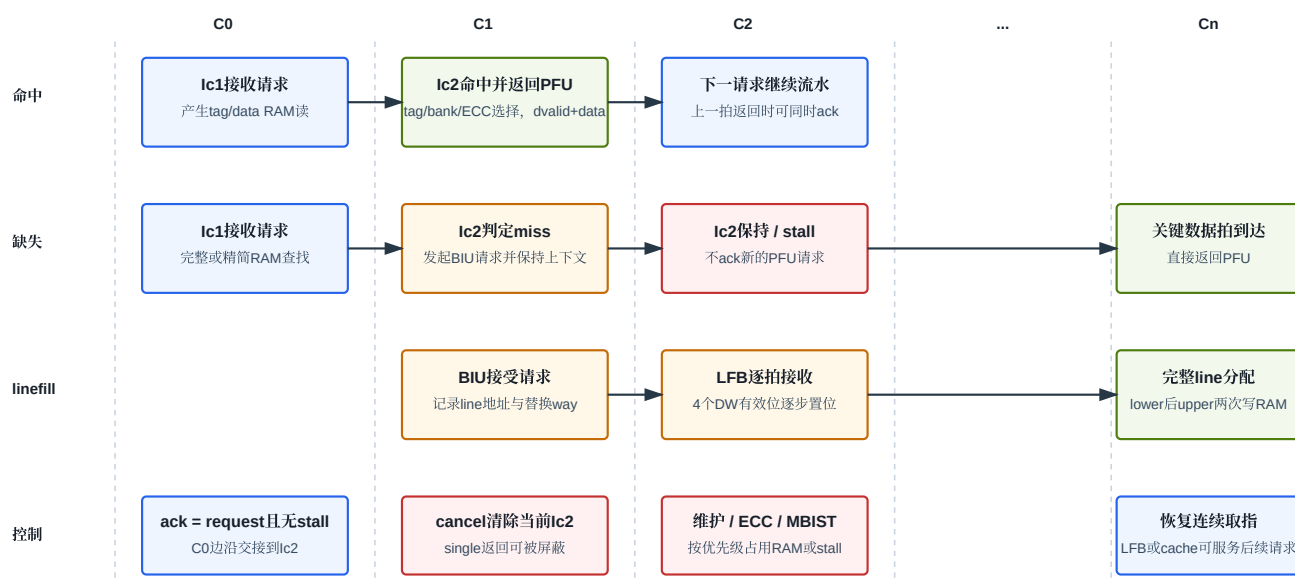
该布局还影响ECC错误定位。data ECC checker首先报告发生错误的物理bank；对于偶数doubleword，bank编号与way编号相同，对于奇数doubleword，bank编号与way编号相反。因此错误来自data RAM且 addr[3]=1 时，ICU必须反转物理bank对应的way编号，才能得到正确的逻辑出错way。tag RAM不使用corkscrew布局，所以tag错误的way编号不需要反转。

重实现必须保持以下规则：

1. full lookup时, bank0和bank1使用同一个 {set index, doubleword index} 地址并行读取。
2. 返回cache数据时, 必须根据命中way与 addr[3] 选择bank, 不能固定把way0等同于bank0。
3. allocation时, way0和way1必须使用表中相反的doubleword-to-bank映射, 并在两拍写入期间保持目标way不变。
4. reduced lookup的单bank使能以及data ECC错误的bank-to-way转换都必须使用同一套corkscrew映射。

3.2 Ic1/Ic2流水线

cm7icu Ic1/Ic2流水线: 命中与缺失路径



每个框左边缘吸附到cycle列边界; "..."表示BIU可变等待时间, 不表示固定延迟。

这张图按 cycle column 对齐展示 ICU 的主 pipeline。C0 中 PFU 的 request 被 ICU 接收时, Ic1 生成 tag/data RAM lookup; C1 中 RAM read data 到达 Ic2, Ic2 同时拿到 PFU 上一拍 request 对应的 memory attributes。若命中, Ic2 在 C1 就可以返回 64-bit data 给 PFU; 若 miss, Ic2 保持 fetch valid, 向 BIU 发请求, 直到 BIU beat 或 LFB data 可满足当前 doubleword。

Stage / box	输入	本级处理	输出给下一级或外部
Ic1 lookup	pfu_icu_req_i、pfu_icu_addr_i、pfu_icu_first_i、cache enable、LFB valid mask、Ic2 stall estimate、MIU lock	屏蔽无效地址; 如果 cache enabled、未预计 stall、未命中 LFB、MIU 未锁住, 则请求 l-cache lookup。根据 first/reduced 策略选择 tag bank 和 data bank enable。	tag RAM address addr[14:5]、data RAM address {addr[14:5], addr[4:3]}、lookup valid metadata。
Ic2 hit	上一拍 Ic1 保存的 fetch address/context、RAM tag/data read data、PFU attributes、LFB state、BIU data beat、ECC status	判断 tag0/tag1 valid/tag compare; 根据 corkscrew bank mapping 选择 data bank; 优先选择 BIU current beat, 其次 LFB, 再次 cache data。	icu_pfu_dvalid_o、icu_pfu_data_o、icu_pfu_bus_err_o; 若 ECC 需要 replay, 下一拍输出 icu_pfu_ecc_err_o。
Ic2 miss/stall	有效 Ic2 fetch 且 cache/LFB/BIU/fake hit 都无法满足	如果 LFB fully allocated、BIU idle、未 cancel、非 MBIST, 则发 linefill; 若 MIU lock 且 cache lookup off, 发 single read。Ic2 fetch 保持 valid, icu_pfu_ack_o 关闭。	icu_biu_req_ic2_o、BIU address/attrs/priv/vf/single; 直到 data valid 或 cancel。
LFB fill	BIU accepted request、BIU data beats、line address、allocation way	记录 line address; 每个 beat 写入对应 doubleword, 置 valid bit; linefill invalidated 或 new request 时清 valid mask; transaction masked 时丢弃返回 data 直到 last。	可直接 forward 当前 beat 给 PFU; 完整 cacheable line 后请求 cache allocation。
Cache write	LFB 四个 doubleword 全部 valid、allocation 未完成、未被 invalidate/ECC block	先写 lower quadword, 再写 upper quadword; 第一次 tag write valid=0, 第二次 tag write valid=1, 避免 RAM 中出现 valid 但 data 不完整的 line。	tag/data RAM write request; allocation completed 后 LFB 可接受下一条 linefill。

Stage / box	输入	本级处理	输出给下一级或外部
Control row	PFU cancel、cache maintenance、ECC invalidate、MBIST lock	cancel 清除 Ic2 fetch；MVA/ECC 维护可 stall fetch；invalidate all 作为低优先级 background RAM user；MBIST 最高优先级访问 RAM。	ack /stall、RAM grant、LFB invalidation、MIU response/error。

这个 pipeline 的设计结论是：ICU 对 PFU 一次只跟踪一个 outstanding fetch response，但 hit path 可达到每 cycle 一个 64-bit response，因为同一拍返回 data 时可接受下一笔 request。miss path 会关闭 `icu_pfu_ack_o`，直到当前 Ic2 fetch 被 data valid 满足或被 PFU cancel。

流水线中的保持规则必须按事务理解，而不是把Ic2当成固定一拍stage。Ic1请求只有在 `ack` 成立时才交给Ic2。cache hit时，Ic2一拍完成并输出 `dvalid`；cache miss时，同一份地址、权限、vector fetch标记和属性一直保持，直到 BIU/LFB提供所需双字或PFU发出cancel。stall期间不能用新的PFU地址覆盖这些上下文。维护和MBIST造成的stall同样只阻止新请求进入，不会丢失已经接管的Ic2 fetch。

3.3 查找策略

ICU 的 lookup policy 是为了兼顾性能和 RAM 访问功耗：

Fetch type	Tag RAM access	Data RAM access	使用场景
full lookup	读 way0/way1 tag	读 bank0/bank1 data	first fetch、新 cache line、force 后 fetch，或 cache/LFB/maintenance 使顺序 way 信息失效。
small lookup	不读 tag	读两个 data bank	first fetch 后一拍，上一拍 hit/miss 结果还不能及时控制当前 tag read；读取两个 data bank 足够在 Ic2 选择。
reduced sequential lookup	不读 tag	只读预测需要的 data bank	已知顺序 fetch 属于上一条 first fetch 命中的 way，且未被 cache write/maintenance/ECC/MBIST 破坏。
no lookup	不访问 RAM	不访问 RAM	cache disabled、estimated stall、Ic1 预计 LFB hit、MIU lock 阻止 normal cache access。

`pfu_icu_first_i` 必须在 reset 后第一笔 accepted fetch、跨 cache line fetch、force 后 fetch 或 PFU 不能保证顺序 way 信息时置位。ICU 内部还会通过 `force_nxt_first` 在 cache 被写入、maintenance、ECC invalidate、allocation 或 MBIST 后强制下一次 lookup 回到 full lookup。

3.3.1 first 标记的设计语义

`first` 表示当前fetch不能继承上一笔fetch记住的“这条cache line是否命中、命中哪个way”信息。ICU必须把它当作新的顺序取指区间起点，重新读取way0和way1的tag，并准备两个data bank。完成这次full lookup后，ICU会记录是否真正从I-cache取得数据以及命中的way，供同一32-byte cache line内后续 `first=0` 的顺序fetch使用。

因此，`first` 描述的是取指地址之间的连续性和cache定位信息是否仍可信，不是数据返回顺序：

- `first=1` 不代表cache miss。重新查找后既可能hit，也可能miss。
- `first=1` 不是linefill第一个beat。BIU beat顺序由linefill地址和 `data_last` 管理，与该标记不同。
- `first=0` 不表示“不查cache”，而是允许ICU使用small lookup或reduced lookup，省略已经不需要的tag/data bank访问。
- `first` 也不只用于reset第一笔取指。程序运行中每次离开已知顺序区间，都可能重新置位。

下面是同一条32-byte cache line内顺序取指的例子。假设每次PFU请求一个64-bit doubleword：

Fetch address	所属32-byte line	first	ICU行为
0x00001000	0x00001000 - 0x0000101F	1	新顺序区间的第一笔，读取两个way的tag和两个data bank，重新确定hit和way。
0x00001008	同上	0	紧跟first fetch，可使用small lookup；进入Ic2后使用上一笔确定的way选择data。
0x00001010	同上	0	顺序命中信息已经稳定，可使用reduced lookup，只读需要的data bank。
0x00001018	同上	0	继续复用同一line的命中way；同时PFU知道下一笔顺序地址将跨line。
0x00001020	0x00001020 - 0x0000103F	1	跨入新的cache line，旧tag/way结论不再适用，重新full lookup。

PFU产生或保留 first 的条件如下。表中的“下一笔”表示当前fetch成功从PF stage推进后，后继fetch应使用 first=1。

触发条件	哪一笔标记为 first	设计原因
当前地址来自非顺序地址选择，例如force、vector或其他程序流重定向	当前fetch	地址不再是上一顺序双字的延续，不能复用旧cache line和way信息。
当前顺序fetch位于32-byte cache line的最后一个doubleword，且没有BTAC taken把后继地址留在已知关系中	下一笔fetch	顺序地址将进入新的cache line。
当前fetch发生BTAC taken，目标不在当前32-byte cache line	下一笔目标fetch	branch target属于另一条line，必须重新比较tag。
BTAC taken目标仍在当前32-byte cache line	下一笔目标fetch 通常保持 first=0	原line的tag和命中way仍然适用；改变line内doubleword offset即可。
PFU发现ICU/TCM chip-select预测与实际地址目标不一致	修正后的下一笔fetch	之前保存的目标选择和cache连续性不可信，需要重新建立查找上下文。
BTAC元数据队列满导致前端预测流被阻塞或重新对齐	恢复后的下一笔fetch	指令流和预测元数据的连续推进被打断，保守地重新建立full lookup基准。
ITCM或DTCM enable配置改变	配置改变后的下一笔fetch	同一个地址可能改由TCM或ICU服务，旧目标选择和I-cache命中信息不能复用。

触发条件	哪一笔标记为 first	设计原因
当前操作不是正常run状态下的普通指令fetch	下一笔普通fetch	vector、控制或特殊取指结束后，需要重新建立普通顺序流。

PFU给出的 `pfu_icu_first_i` 并不是full lookup的唯一来源。即使PFU送来 `first=0`，ICU也必须在以下情况内部提升为“等效first”：

1. 上一次标为first的fetch最终由LFB或当前BIU beat满足，而不是由I-cache命中。此时没有可靠的cache hit way可供后续reduced lookup使用。
2. 前一周期刚接受first fetch，命中way结果还来不及控制下一拍RAM enable。ICU对紧随其后的顺序fetch执行small lookup，即不读tag但读取两个data bank。
3. 自上次可靠full lookup以后发生cache RAM写入，包括linefill allocation、invalidate by MVA、ECC invalidate、invalidate all或MBIST访问。写操作可能改变valid、tag或data，因此下一笔必须重新full lookup。

重实现时可以把最终full-lookup条件理解为：

```
full_lookup_required = PFU.first
                        or 顺序命中way尚未建立
                        or 上次first未由I-cache满足
                        or 自建立顺序基准后cache RAM被写过
```

该伪代码只表达设计语义。实际实现还要保留紧跟first fetch时的small lookup特例，不能把所有等效first都无条件转换成重复tag读取，否则会改变RAM enable和cycle-accurate功耗行为。

三类查找的设计语义如下：

- **full lookup**：当前地址可能位于任意way，也可能完全miss，所以同时读取两路tag和两个data bank。它建立“该顺序取指流是否命中cache、命中哪一路”的新基准。
- **small lookup**：紧跟first fetch的下一笔顺序请求到来时，上一笔命中way还来不及控制当前拍RAM enable。ICU不再重复读tag，但读取两个data bank，等进入Ic2后再用上一笔保存的way选择结果。
- **reduced lookup**：first fetch已经证明同一cache line命中某way，且中间没有RAM写或维护破坏该判断时，只读corkscrew映射后真正需要的一个data bank。该模式是功耗优化，不得改变返回数据或时序。

以下事件会使顺序命中知识失效并强制后续重新full lookup：LFB/BIU而非cache满足first fetch、cache allocation、MVA/ECC/全部失效写tag、MBIST访问，以及其他会写cache RAM的事件。其目的不是“多查一次更保险”，而是防止继续使用写入前记住的way和valid状态。

Ic1 使用一个保守的 `icu_stall_estimate_ic2` 控制 lookup issue。这个 estimate 不尝试覆盖所有实际 stall 情况，而是避免把 Ic2 tag/data 结果放在 Ic1 timing critical path 上。兼容实现允许做同等保守优化，但必须保持外部可见行为：不能 ack 一个无法最终保持/返回/cancel 的 fetch，也不能让 MBIST 因连续 fetch lookup 永久无法进入。

3.4 命中、缺失与数据来源优先级

Ic2 对当前 fetch 的 data source 选择必须按以下优先级：

Priority	Source	Condition	PFU visible behavior
1	BIU current beat	outstanding linefill/single read 返回的 beat 正好满足当前 fe_addr_ic2	同周期 icu_pfu_dvalid_o=1, data 直接来自 BIU; 若 biu_icu_fault_i=1, 同周期 icu_pfu_bus_err_o=1。
2	LFB	LFB line address match 且对应 doubleword valid	icu_pfu_dvalid_o=1, data 来自 LFB; 用于 linefill 尚未 allocation 完成或 non-cacheable burst 的后续 doubleword。
3	fake hit	上一拍ECC error正在处理且本拍 lookup data不可信	屏蔽cache data, icu_pfu_dvalid_o=1 但data不可使用; 下一拍 icu_pfu_ecc_err_o=1, PFU必须replay。
4	I-cache RAM	tag hit或reduced lookup, 且memory attribute cacheable, 并且没有fake hit 覆盖	icu_pfu_dvalid_o=1, data来自selected data bank; ECC error如需报告, 下一拍通过 icu_pfu_ecc_err_o。
5	miss	以上均不满足	不返回 data; Ic2 保持 fetch valid, 关闭 ack, 发起或等待 BIU/LFB。

3.4.1 Cache hit数据是否经过LFB

不经过。I-cache data RAM、LFB和BIU current beat是三个并列的数据候选源，它们在Ic2进入同一个返回数据选择器，而不是串联成“I-cache -> LFB -> PFU”。I-cache命中时，corkscrew选择逻辑先从bank0/bank1中选出命中way对应的64-bit doubleword，然后该数据直接驱动PFU返回路径；这个过程既不读取LFB，也不修改LFB的地址、data或valid状态。

I-cache hit: data RAM -> corkscrew bank选择 -> Ic2返回选择器 -> PFU

LFB hit: LFB已有效DW -----> Ic2返回选择器 -> PFU

BIU hit: BIU当前beat -----> Ic2返回选择器 -> PFU

优先级: BIU > LFB > I-cache

三条路径与LFB状态更新的关系如下：

当前PFU数据来源	是否先进入LFB再返回	是否更新LFB	设计语义
I-cache data RAM hit	否	否	从命中data bank直接返回PFU；LFB完全不在该数据路径上。
BIU current beat, 普通linefill	否	有效且未被屏蔽时写入	当前beat可同拍直接forward给PFU，并同时写入LFB对应doubleword槽；“同时写入”不是“先写后读”。
BIU current beat, single read	否	否	cache关闭或MIU lock场景下直接返回PFU，不作为普通linefill保存在LFB。
stored-LFB hit	数据此前已在较早拍进入LFB	本次读取不新增数据	从LFB已置valid的doubleword槽返回PFU。
LFB allocation到I-cache	不属于PFU hit 返回路径	读取LFB并写 cache RAM	完整line收齐后执行的后台写入，数据方向是LFB到I-cache，与cache hit方向相反。

LFB优先于I-cache的原因是，正在linefill的cache line可能只有部分数据已经返回，或者尚未完成两拍allocation。此时I-cache RAM中同地址位置可能仍是旧line，也可能刚完成部分写入，不能用它代替LFB中的最新linefill数据。因此只要当前地址命中LFB有效槽，返回选择器就使用LFB；这表示多个候选源之间的覆盖优先级，不表示cache hit数据会被送进LFB。

重实现时，I-cache hit不得产生LFB data write、LFB valid-bit更新或LFB地址更新。只有被接收的BIU普通linefill数据拍才能填充LFB数据槽；从LFB到I-cache的allocation则是独立的后台路径。

Bus fault 只来自 BIU path，不能来自 cache hit。ECC error 只来自 tag/data RAM lookup 或 maintenance lookup，不能与同一 fetch 的 bus fault 同时作为 PFU fetch error 报告。PFU cancel 后，ICU 不应再为该 canceled fetch 输出 ECC error。

cache hit不是只比较tag。完整判定顺序是：本次Ic1 lookup的RAM读结果有效；memory attribute为cacheable；目标way valid；容量mask后的tag相同；该index/way没有被IEBR或正在处理的ECC错误屏蔽。reduced lookup是例外，它复用first fetch保存的命中way，不重新读tag，但仍要求当前属性cacheable。两个tag way正常情况下不得同时命中；若ECC使tag不可信，错误路径优先接管。

`fake hit` 不是第五种真实数据源。它在“上一拍已注册ECC错误，但新lookup结果同时到达”的冲突窗口中把miss暂时压掉，并给PFU一个零/无意义占位数据，下一拍必须用ECC错误让PFU replay。兼容实现可以改变内部占位值，但不能让该数据被提交，也不能漏掉下一拍错误通知。

3.5 LFB与BIU请求行为

LFB保存一个linefill stream的line地址、当前返回doubleword位置、4个64位槽、4位valid mask、allocation way、2位allocated-qword mask，以及 `biu_trans_masked`。它不是FIFO队列，也不能同时跟踪两条cache line；任一时刻最多有一个linefill/single总线事务。

cacheable linefill从critical doubleword地址开始，总共返回4拍。每收到一拍，LFB槽索引按line内doubleword编号加1并在2位范围内回绕，所以无论请求从DW0、DW1、DW2还是DW3开始，四拍后都能覆盖整条line。non-cacheable burst不回绕，只从critical doubleword读到32字节边界，因此长度是1到4拍。

3.5.1 LFB何时可以向PFU返回数据

LFB不需要等4个doubleword全部有效才向PFU返回。对当前Ic2 fetch，只要同时满足以下条件，就构成stored-LFB hit：

1. 当前Ic2中确实有一笔有效fetch。
2. fetch地址与LFB记录的地址属于同一条32字节cache line，即line address匹配。
3. fetch所需doubleword对应的LFB valid bit为1。

这里检查的是“当前需要的一个doubleword是否有效”，不是4位valid mask是否等于 `1111`。4位valid mask中，bit0、bit1、bit2、bit3分别表示DW0、DW1、DW2、DW3是否已经从BIU返回。例如PFU等待DW2时：

```
LFB valid mask = 4'b0100
                  ^
                  DW2已经有效
```

即使DW0、DW1和DW3尚未返回，DW2已经足以满足当前fetch，ICU可以立即从LFB的DW2槽读取64-bit数据并向PFU输出 `icu_pfu_dvalid_o`。因此LFB同时具有两种不同粒度的完成条件：

使用目的	所需条件	是否要求LFB填满
满足当前PFU fetch	当前地址的line匹配，且目标DW valid=1	否，只需要一个目标DW有效。
BIU current-beat forward	当前BIU beat地址就是PFU等待的DW	否，数据可在写入LFB寄存器的同一拍直接返回。
向I-cache执行 allocation	4个DW valid全部为1，且cacheable、cache enabled、目标way可用、没有fault/invalidate	是，必须先得到完整32字节line。

以cacheable miss从DW2开始为例，BIU按DW2、DW3、DW0、DW1的顺序回绕返回：

BIU返回进度	采样后的LFB valid mask	PFU可使用的数据	是否允许cache allocation
DW2正在当前拍返回	0100	DW2可走BIU current-beat forward；采样后也可从LFB读取	否
随后返回 DW3	1100	DW2、DW3	否
随后返回 DW0	1101	DW0、DW2、DW3	否
最后返回 DW1	1111	整条line的所有DW	若其他分配条件也满足，则开始两拍allocation

non-cacheable burst进一步说明“向PFU返回”不能依赖LFB填满：它只从critical doubleword读到32字节line边界，不回绕补齐前面的doubleword，因此可能永远不会得到 1111，但其中已经有效的目标DW仍然必须能够返回PFU。non-cacheable数据不会执行正常I-cache allocation。

前递有两条路径。BIU current-beat forward在目标数据当前拍刚从BIU返回时使用，数据不等待LFB寄存器更新；stored-LFB forward在目标数据已于较早拍写入LFB后使用。两者都不要求整条line填满，前者在返回选择器中优先于后者。

BIU request分两类：

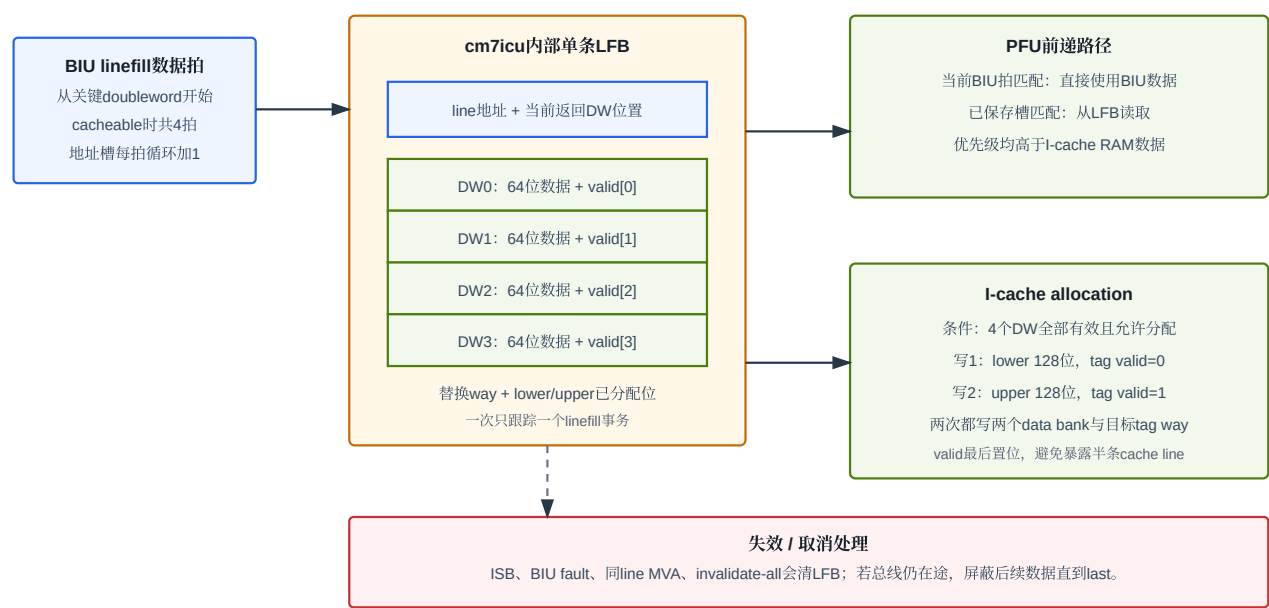
Request type	Trigger	BIU expected data length	Allocation
linefill request	lc2 miss、未cancel、LFB allocation 已完成、BIU idle、MIU 未锁住	cacheable request 返回 4 beat；non-cacheable request 返回从critical doubleword 到 line boundary 的 1 到 4 beat	只有 cache enabled、lookup enabled、attribute cacheable、且至少一个 way 未被 block 时才写入 cache。
single request	lc2 miss、cache lookup off、MIU lock active、BIU idle	1 beat	不使用正常 LFB allocation；用于 MIU lock/cache off 下允许 fetch 前进。

LFB invalidation条件包括Instruction Synchronization Barrier (ISB, 指令同步屏障) retire、BIU fault data、invalidate by MVA命中同一line、invalidate all start。RTL对ISB的实际行为是无条件清当前LFB, 不只清non-cacheable数据。若LFB失效或single事务被cancel时BIU已有outstanding transaction, ICU将 `biu_trans_masked` 置位, 丢弃后续beat, 直到 `data_last` 到达后才允许新BIU请求。这样旧控制流或维护之前的数据不会重新进入PFU、LFB或cache。

LFB部分有效时仍可服务后续PFU请求, 但LFB invalidation会清除这些valid信息。第一种前递路径是BIU current-beat forward, 第二种是stored-LFB forward; 其详细条件见3.5.1节。RTL断言保证同一fetch不会同时命中当前BIU拍和旧LFB槽。

3.6 Cache RAM写入与allocation

Linefill Buffer (LFB) 与I-cache写入结构



这张图把 data path 的存储结构拆开看。Fetch address 的 `addr[31:11]` 是 tag compare 主体, `addr[14:5]` 是 set index, `addr[4:3]` 选择 cache line 中的 64-bit doubleword。I-cache 是 2-way, tag RAM 每个 way 保存 `{ECC[6:0], valid, tag[20:0]}`; data RAM 每个 bank 保存 `{ECC[7:0], data[63:0]}`。由于 data RAM 使用 corkscrew 映射, way 与 bank 的关系由 doubleword index bit `addr[3]` 翻转: 偶数 doubleword 时 way0 在 bank0、way1 在 bank1; 奇数 doubleword 时 way0 在 bank1、way1 在 bank0。

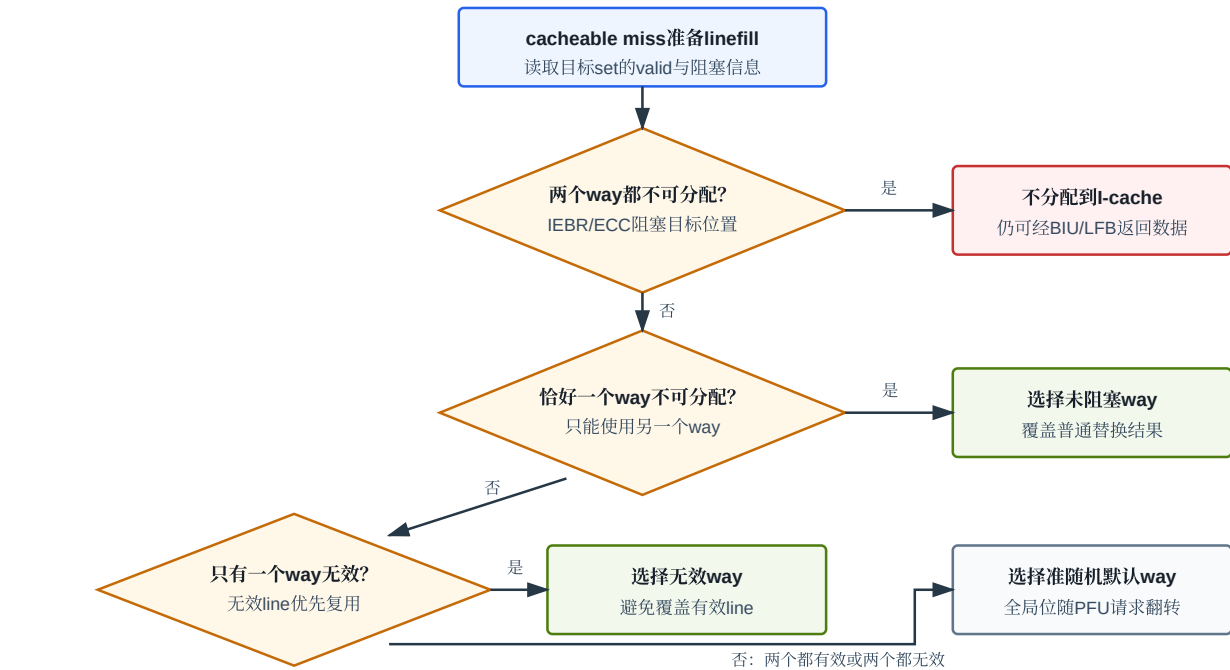
读图先看左侧BIU到LFB的数据流。linefill请求被ack时, ICU锁存line地址和替换way, 并清4位valid mask; 每个有效返回拍写一个DW槽。当前critical拍可直接走上方PFU前递路径, 后续fetch也可读取已经置valid的槽。图下方红框表示失效不会终止总线协议本身: 若事务已经被BIU接受, ICU只是屏蔽数据并等待last收尾。

再看右下allocation路径。只有四个doubleword全部valid且该line允许cache分配时, ICU才分两次写data RAM。第一次并行写lower 128位, 同时写tag但valid保持0; 第二次并行写upper 128位, 同时把tag valid置1。这个顺序保证任何观察到valid line的lookup都不会读到只写了一半的数据。若MVA正在处理同一line、IEBR/ECC阻塞目标位置、BIU返回fault或LFB被清除, 未完成allocation立即标记为不可继续。

Replacement way 选择规则如下:

Condition	Allocation way
way0 被 IEBR/ECC block, way1 未被 block	way1
way1 被 IEBR/ECC block, way0 未被 block	way0
两个 way 都被 block	no allocation
只有 way0 valid, way1 invalid	way1
只有 way1 valid, way0 invalid	way0
两个 way 都 valid 或都 invalid	使用 pseudo-random/default way, RTL 中该 default 在 PFU request 上翻转。

I-cache替换way选择流程



该策略不是LRU；没有为每个set保存“最近使用”状态。

该流程先处理可靠性约束，再处理普通替换策略。若两个way都被IEBR/ECC阻塞，本次BIU请求会被改成不可分配，数据仍可返回PFU但不写cache；若只阻塞一路，只能选择另一路。没有阻塞时，优先使用唯一无效way，避免无必要地覆盖有效line；两个way同为有效或同为无效时才使用全局准随机默认位。

这里没有Least Recently Used (LRU，最近最少使用)信息，也没有每个entry或每个set的小状态机。整个ICU只有一个共享的 `default_alloc_way0` 寄存器：复位值0代表默认选择way1，值1代表默认选择way0。任意set的PFU request都会使它翻转，cache hit也会翻转；它不是在“某个set发生替换”时才更新。因而某个set下次miss选择哪一路，会受到此前访问其他set的请求影响。

replacement候选在miss的第一个有效lookup周期确定；如果后续需要等待BIU接受linefill请求，ICU保存该选择。BIU对可分配linefill给出ack时，最终目标way被锁存到LFB上下文，后续四个返回doubleword和两拍cache RAM写入必须一直使用同一个way，不能因为全局默认位继续翻转而中途换way。

重写如果追求cycle-accurate兼容，应保留“单个全局位、随每个PFU request翻转、只在两way valid相同时作为默认候选、linefill开始后锁存目标way”这四项行为。只做架构功能模型时可以换成其他公平策略，但必须明确这不再是周期级等价实现，并且仍不得选择被阻塞way，也不得在存在唯一invalid way时覆盖有效line。

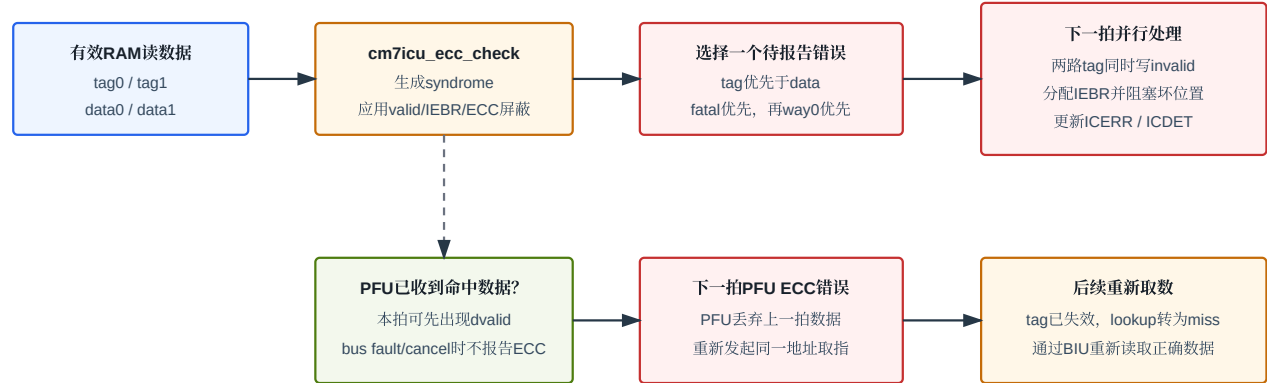
3.7 ECC与IEBR

当 `ICACHE==1` 且 `CACHEECC==1` 时，ICU 实例化 `cm7icu_ecc_check`。ECC checker 同时检查 tag0、tag1、data0、data1。tag ECC 的 syndrome 需要把 tag、valid、masked index 组合成 32-bit check data；data ECC 对 64-bit instruction data 检查 8-bit ECC。当前 RTL 中 syndrome qualification 使用 `~dpu_icu_cacr_ecc_i`，因此兼容实现必须在该信号为 1 时屏蔽 ECC 检测/上报。

ECC classification 规则：

Rule	Required behavior
tag/data valid qualification	只有本拍 RAM read data 对 lookup 或 maintenance 有效时才检查；已被 IEBR mask、fake hit mask、invalid tag mask 的 bank 不报告 error。
fatal classification	tag ECC 中落在 index field 的单 bit error 也按 fatal 处理，因为它暗示 RAM decoder/index path 错误。
multiple-error priority	tag error 优先于 data error；fatal 优先于 correctable；way0 优先于 way1。data bank 到 way 的映射必须考虑 corkscrew。
PFU report timing	fetch data 先以 <code>icu_pfu_dvalid_o</code> 返回；若该 data 因 ECC 不可信， <code>icu_pfu_ecc_err_o</code> 在下一拍报告。
ECC invalidate	ECC error注册后一拍发起tag invalidate，在出错index同时清除way0和way1的tag entry。这样即使错误来自tag/index路径，也不会保留同set的可疑命中。
IEBR allocation	ECC error 注册后尝试写入 <code>iebr0</code> 或 <code>iebr1</code> ；locked EBR 不会被自动覆盖。
System report	<code>icu_ext_icerr_o</code> 汇总 EBR valid/locked、fatal/tag-data/way/location/allocated register； <code>icu_ext_icdet_o</code> 指示 data fatal、data correctable、tag fatal、tag correctable 分类。

cm7icu ECC检测、报告与恢复流程



"correctable"是ECC分类，不表示ICU把数据原地修正后继续使用；当前RTL采用失效并重取。
若已注册ECC错误与新lookup重叠，ICU可产生fake hit占位，下一拍仍要求PFU replay，不能提交占位数据。

图从左到右展示错误处理主链。`cm7icu_ecc_check` 只检查前一拍确实被enable读取的bank，并先应用cache-size mask、tag valid、IEBR和全局ECC控制。syndrome非零时，当前拍选出一个优先错误并寄存；下一拍ECC invalidate、IEBR分配和系统错误输出并行发生。错误位置包含index、way以及data doubleword位置；tag错误没有有意义的doubleword位置，所以记录时清掉低两位。

下方是PFU恢复链。命中数据可能在检测拍先以 `dvalid` 到达PFU，下一拍 `icu_pfu_ecc_err_o` 使PFU丢弃该数据并重新请求。重新lookup时tag已经失效，因此正常转为BIU miss并重取。ECC分类中的“correctable”只表示syndrome属于单比特可纠正类别；本ICU没有把错误data修正后继续执行，而是统一失效并重取。

IEBR 字段格式按 32-bit register 表示：

Bits	Field	Meaning
[31:30]	sw_def	software-defined bits, PPB 写入时保留给软件。
[29:18]	reserved	读回为 0 或被 mask。
[17]	fatal	1 表示 fatal, 0 表示 correctable。
[16]	bank	0 表示 tag, 1 表示 data。
[15]	reserved	0。
[14]	way	0 表示 way0, 1 表示 way1。
[13:2]	location	[11:2] 是 masked index, 低位包含 doubleword location; tag error 会清掉无意义低位。
[1]	locked	1 表示软件锁定, 自动 ECC allocation 不可覆盖。
[0]	valid	1 表示该 EBR entry 有效并参与 ECC mask/allocation block。

两个IEBR都valid且未locked时，自动allocation使用preferred register，并在每次ECC detection后翻转偏好，避免连续错误永远覆盖同一个EBR。IEBR有效后会mask同index/way的后续ECC error，并阻止LFB allocation写入该way/index。两个IEBR都locked时，新错误不会覆盖它们；系统报告会标识“错误未分配到可写IEBR”，但ECC invalidate仍执行。

`icu_ext_icdet_o[3:0]` 位义依次为：data fatal、data correctable、tag fatal、tag correctable。若同拍有多个syndrome，ICDET可以反映多个类别，但IEBR/ICERR只按前述优先级记录一个主错误。`dpu_icu_cacr_ecc_i=1` 在当前RTL中会屏蔽syndrome qualification；虽然端口名容易被理解为“ECC enable”，重写必须服从实际高电平屏蔽语义。

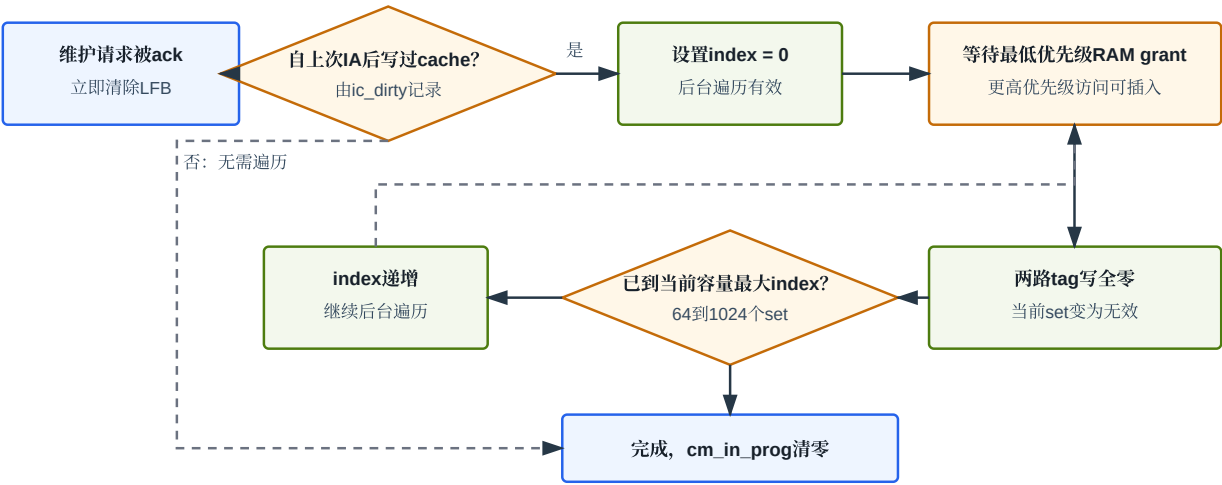
3.8 Cache维护

ICU 支持两类 cache maintenance request。接口上 request 来自 `dpu_icu_cm_req_wr_i`、operation 来自 `dpu_icu_cm_ia_wr_i`，集成中这些信号由 PPB 侧寄存器连接。

Operation	Ack semantics	Completion behavior	Fetch interaction
invalidate by MVA	仅当 MVA FSM idle 时 ack; ack 后该 MVA 操作保证由 ICU 接管并完成。	先 tag lookup; 若命中, 则写 tag valid=0; 若 LFB line 匹配, 也 invalidate LFB; 若 lookup 期间发现 ECC, 则由 ECC invalidate 接管。	MVA FSM 非 idle 会 stall PFU ack; lookup/write 也参与 RAM arbitration。

Operation	Ack semantics	Completion behavior	Fetch interaction
invalidate all	request 总是可 ack。若 cache 自上次 invalidate all 后没有 allocation 写入，可不做后台遍历。	如果 cache dirty，后台从 lowest index 递增到当前 cache size 的 max index，对两个 tag way 写 invalid。每次 start 都 invalidate LFB。	后台 invalidate all 是最低优先级 RAM user，fetch、ECC、MVA、allocation 可优先进行； <code>icu_lsu_cm_in_prog_o</code> 在后台进行期间有效。

Invalidate-all后台遍历流程



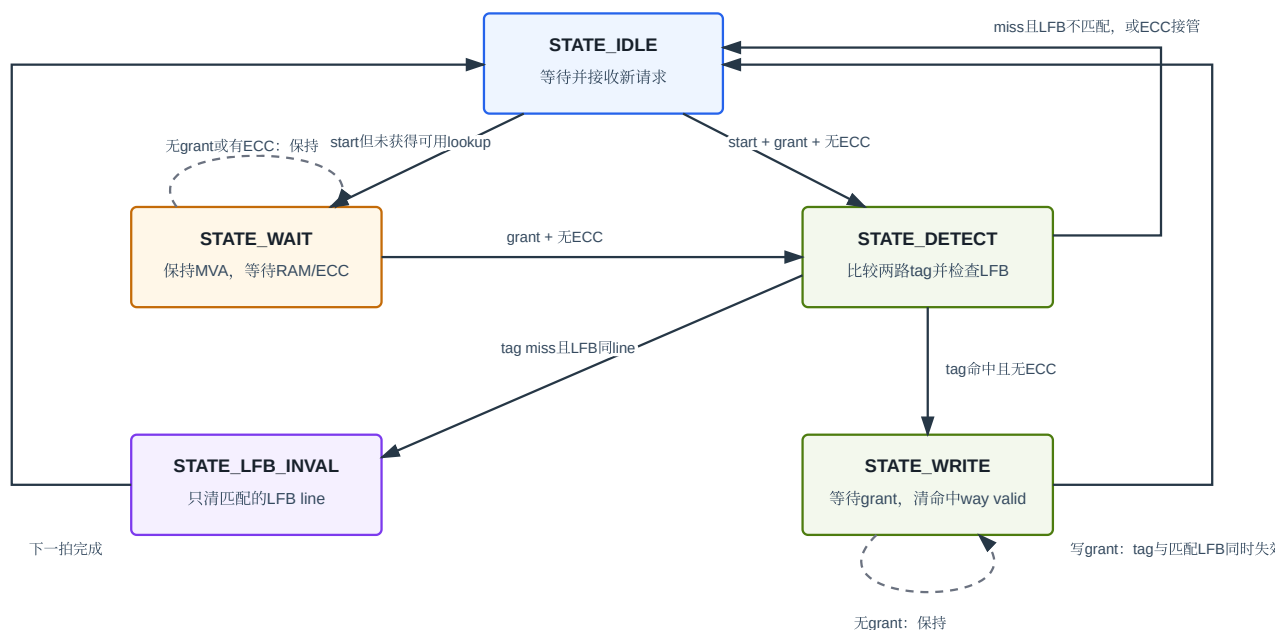
Invalidate-all不是前台阻塞状态机，而是 `cm_ia_val` 加 `cm_ia_addr` 构成的后台walker。请求在入口立即ack并无条件清LFB；只有 `ic_dirty=1` 才把walker置为有效。`ic_dirty` 复位为1，任一cache allocation也置1，接收invalidate-all时清0，所以复位后的第一次invalidate-all一定真正遍历RAM，而连续两次中间没有allocation时第二次可省略遍历。

walker从index 0开始。每次获得最低优先级 `ic_ia_grant` 时，同时把两路tag写成全零并把index加1；没有grant时 `cm_ia_val` 和当前index必须保持，不得跳项。到当前容量对应的最大index并完成写入后，`cm_ia_val` 清零。整个进行期间 `icu_lsu_cm_in_prog_o` 持续为1，即使若干周期没有grant，也不能短暂报告完成。

Invalidate by MVA FSM 的行为合同：

Current state	Event / condition	Next state	Action
STATE_IDLE	start 且 tag lookup grant 且无 ECC	STATE_DETECT	捕获 MVA，读 tag。
STATE_IDLE	start 但未完成可用 lookup 或有 ECC	STATE_WAIT	捕获 MVA，等待 ECC/port 可用。
STATE_WAIT	tag lookup grant 且无 ECC	STATE_DETECT	读 tag。
STATE_WAIT	否则	STATE_WAIT	保持 MVA。
STATE_DETECT	tag0 或 tag1 hit 且无 ECC	STATE_WRITE	记录命中 way，准备写 invalid tag。
STATE_DETECT	cache miss 但 LFB 同 line 有 valid/in-progress data	STATE_LFB_INVAL	仅 invalidate LFB。
STATE_DETECT	miss 且 LFB 不匹配，或 lookup 被 ECC 接管	STATE_IDLE	操作完成。
STATE_WRITE	invalidate write grant	STATE_IDLE	清除命中 way valid，并同步 invalidate matching LFB。
STATE_WRITE	no grant	STATE_WRITE	等待 RAM grant。
STATE_LFB_INVAL	always	STATE_IDLE	LFB invalidation 完成。

cm7icu invalidate-by-MVA状态跳转图



图中省略STATE_IDLE无start自环；状态编码可替换，但进入、保持、写入与完成语义必须保持。

这张状态跳转图描述 `cm_im_state` 的主控制流，表格仍是实现合同的完整来源。`STATE_IDLE` 是唯一能接受新的 invalidate by MVA request 的状态；如果 request 同拍拿到 tag lookup grant 且没有 ECC 接管，就直接进入 `STATE_DETECT`，否则进入 `STATE_WAIT` 保存 MVA 并等待 tag port 可用。`STATE_WAIT` 不改变目标 MVA，只在 grant 且无 ECC 时前进；因此它会通过 ICU stall 阻止 PFU 把新的 fetch 交错到同一维护窗口里。

`STATE_DETECT` 在 tag RAM read data 有效后判断三条路径：tag hit 进入 `STATE_WRITE`，等待 tag write grant 后清除命中 way 的 valid bit；tag miss 但 LFB line 匹配且 LFB 有 valid 或 in-progress data 时进入 `STATE_LFB_INVAL`，只清 LFB 后完成；tag miss 且 LFB 不匹配，或者 lookup 被 ECC invalidate 接管时直接回到 `STATE_IDLE`。图里没有把所有低优先级保持条件都展开成大箭头：`STATE_WAIT` 的 no-grant/ECC 保持、`STATE_WRITE` 的 no-grant 保持、`STATE_IDLE` 的 no-start 保持都在表格中定义，重写实现必须保持这些自环语义。

逐状态实现要求：

- **STATE_IDLE**：唯一允许ack新MVA请求的状态。收到请求时捕获按32字节对齐后的MVA高位，并尝试在同拍申请tag read。无请求时不发MVA RAM访问，不改变已保存MVA，也不因MVA维护stall PFU。请求已ack后不能因后续RAM繁忙而丢弃。
- **STATE_WAIT**：请求已经提交，但tag端口当拍被MBIST/ECC等更高优先级用户占用，或当前存在ECC检测结果。该状态反复请求两路tag read并保持MVA，禁止新PFU请求进入Ic2。只有获得grant且本拍没有新ECC时才能进入 `STATE_DETECT`；等待期间不写tag、不清LFB、不报告维护完成。
- **STATE_DETECT**：消费上一拍tag read结果，分别检查way0/way1的valid、masked tag compare和IEBR/ECC屏蔽，同时比较LFB line地址。命中cache且无ECC时保存命中way并转 `STATE_WRITE`；cache miss但LFB仍有该line数据或linefill正在进行时转 `STATE_LFB_INVAL`；两者都不命中时完成。若tag read发现ECC，通用ECC路径会失效整个set，本MVA操作直接结束，不能重复写一个不可信命中way。
- **STATE_WRITE**：持续申请目标ways的tag write，写入全零tag/ECC合法零码以清valid。没有grant时保持命中 way、MVA和状态，PFU继续stall；grant成立的同拍如果LFB也是该line，同时清LFB，然后回 `STATE_IDLE`。该状态不修改data RAM，因为valid清零已经足以让旧data不可见。
- **STATE_LFB_INVAL**：只处理“cache tag miss但LFB同line”的情况。进入后产生一次LFB invalidation，若BIU事务仍在途则启动transaction mask，下一拍无条件回 `STATE_IDLE`。它不访问tag/data RAM，也不等待BIU真正返回last才允许维护状态机本身结束；总线busy由独立mask状态继续跟踪。

TBD: reimplement blocker: 当前解包中的 `cm7icu_decl.v` 是0字节, `STATE_IDLE`、`STATE_WAIT`、`STATE_DETECT`、`STATE_WRITE`、`STATE_LFB_INVAL` 的具体bit encoding未在可读ICU source中定义。功能重写可用任意安全编码; 若目标是netlist/scan/trace完全兼容, 需要补齐原始declaration文件。

3.9 MBIST与RAM仲裁

ICU 对 I-cache RAM 的 grant 优先级从高到低为:

- 1. MBIST access。
- 2. ECC-triggered invalidate。
- 3. invalidate by MVA。
- 4. normal lookup。
- 5. LFB allocation。
- 6. invalidate all。

MBIST lock ack 只有在 ICU 没有 linefill in progress、没有未完成 LFB allocation、没有可能的新 ECC error、没有已注册 ECC error 时才能给出; MBISTALL 模式可以对全部 arrays 做同一 transaction。MIU read data 在 request 后两级 pipeline 返回。若 ICU 在接受 MIU lock 的同一条件下仍检测到 normal cache user 请求, `icu_miu_err_o` 报告冲突。

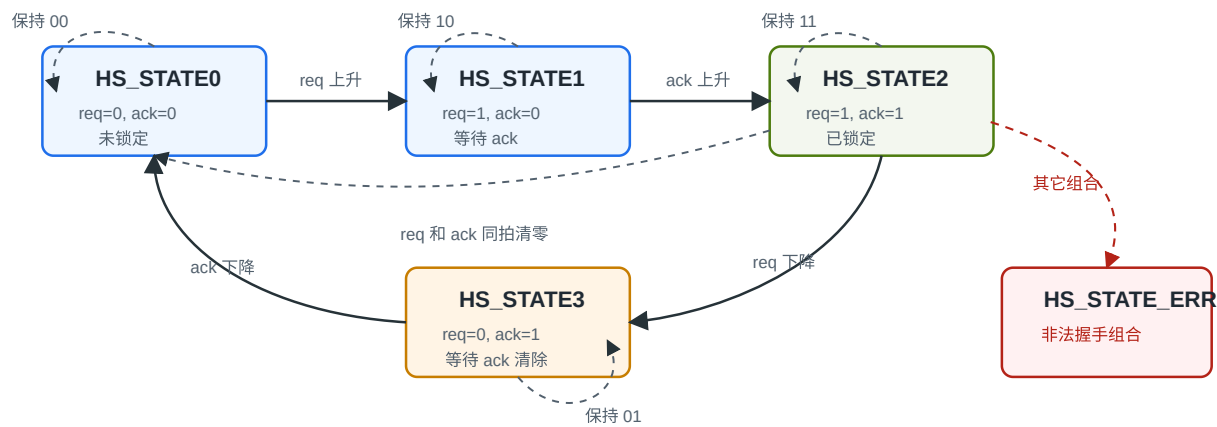
MBIST 使用同一组 RAM 端口, 因此会阻止 normal lookup/allocation/maintenance。为了避免持续 fetch 饿死 MBIST, Ic1 在 MIU lock request 存在时不发 normal lookup; 如果 cache lookup 已关闭且 MIU lock active, ICU 允许 fetch 通过 single BIU request 前进, 而不是使用 LFB/cache。

MIU到ICU lock handshake使用four-state request/acknowledge protocol。该状态机在 `cm7_miu_icu.v` 接口约束中描述, 定义 `miu_icu_lock_req_i` 和 `icu_miu_lock_ack_o` 的合法相位; 它是接口协议检查状态机, 不是 `cm7icu` 内部RAM控制状态编码:

Current state	Event / condition	Next state	Action
<code>HS_STATE0</code>	<code>req=0, ack=0</code>	<code>HS_STATE0</code>	未锁定, MIU 不能访问 ICU RAM。
<code>HS_STATE0</code>	<code>req=1, ack=0</code>	<code>HS_STATE1</code>	MIU 发起 lock request, 等待 ICU ack。
<code>HS_STATE0</code>	<code>req=1, ack=1</code>	<code>HS_STATE2</code>	request 和 ack 同拍建立, 直接进入 locked。
<code>HS_STATE1</code>	<code>req=1, ack=0</code>	<code>HS_STATE1</code>	request pending, ICU 尚未允许 RAM 访问。
<code>HS_STATE1</code>	<code>req=1, ack=1</code>	<code>HS_STATE2</code>	lock 建立, MIU 可以按 MBIST interface 发起 RAM 访问。
<code>HS_STATE2</code>	<code>req=1, ack=1</code>	<code>HS_STATE2</code>	保持 locked。
<code>HS_STATE2</code>	<code>req=0, ack=1</code>	<code>HS_STATE3</code>	MIU 释放 request, 等待 ICU 拉低 ack。
<code>HS_STATE2</code>	<code>req=0, ack=0</code>	<code>HS_STATE0</code>	request 和 ack 同拍释放, 回到未锁定。
<code>HS_STATE3</code>	<code>req=0, ack=1</code>	<code>HS_STATE3</code>	release pending, 等待 ack 清除。
<code>HS_STATE3</code>	<code>req=0, ack=0</code>	<code>HS_STATE0</code>	release 完成。

Current state	Event / condition	Next state	Action
any legal state	其它 req/ack 组合	HS_STATE_ERR	协议错误；接口 assertion 应报错，功能实现不应依赖该路径恢复。

MIU 到 ICU lock handshake 状态跳转图



读这张状态跳转图时，先按正常lock flow看 HS_STATE0 -> HS_STATE1 -> HS_STATE2。MIU拉高request后，ICU等待linefill结束、LFB不存在未完成allocation、且没有可能或已注册ECC，再拉高ack。ack条件本身不等待所有普通lookup/maintenance请求消失；若lock建立时仍有这些cache user请求，icu_miu_err_o 报告冲突。进入 HS_STATE2 后，MIU拥有RAM访问窗口，普通lookup/allocation/maintenance必须让出端口。释放时走 HS_STATE2 -> HS_STATE3 -> HS_STATE0，也允许req和ack同拍清零直接回 HS_STATE0。HS_STATE_ERR 只表示接口违例。

逐状态协议要求：HS_STATE0 中MIU没有所有权，read/write必须为0；HS_STATE1 中request必须保持为1，ICU等待linefill结束、未分配LFB清空以及可能/已注册ECC消失后才能拉高ack；HS_STATE2 中req/ack都保持1，MIU可以发单周期read或write，不能同拍同时读写；HS_STATE3 中request已撤销但ack尚未撤销，MIU不能开始新事务，ICU完成尾部响应后拉低ack；HS_STATE_ERR 只用于assertion发现非法四相跳转。MBIST read从MIU输入到RAM访问再到 icu_miu_rdata_o 经过mb0、mb1、mb2两级响应跟踪，重写不能把read data提前到请求同拍。

3.10 复位、cache enable与首次使用

ICU复位只清控制状态，不自动遍历l-cache RAM。当前通用RAM模型的cache RAM本身也配置为不随core reset清零，因此复位后tag/data内容可能是旧值或未知值。安全性来自两个约束：cm7ppb 中的Cache Control Register instruction-cache enable位复位为0，cache关闭时ICU不做正常tag lookup；软件在第一次置位cache enable前必须执行invalidate-all，使所有tag valid确定为0。

推荐且与本RTL匹配的首次启用顺序是：

- 1. 保持 CCR.IC=0，此时取指经BIU/LFB返回，不产生有效cache allocation。
- 2. 发起invalidate-all并等待 icu_lsu_cm_in_prog_o 清零。因为 ic_dirty 复位为1，这次操作一定从index 0遍历到当前容量最大index。
- 3. 需要时配置ECC/IEBR策略。
- 4. 置位 CCR.IC=1。后续cacheable first fetch执行full lookup；第一次miss收齐line后才产生首个valid cache entry。

关键复位状态如下：

状态	复位值	复位后的设计含义
fe_val_ic2	0	没有在途PFU响应。
lf_in_prog / single_in_prog_ic2	0 / 0	没有在途BIU事务。
LFB valid mask	0000	所有LFB data槽不可被前递。
LFB allocated mask	11	不存在等待写cache的lower/upper部分。
biu_trans_masked_ic2	0	没有待丢弃的旧总线事务。
MVA state / IA valid	STATE_IDLE / 0	没有维护操作进行。
ic_dirty	1	强制复位后第一次invalidate-all真正遍历。
IEBR0/1 valid、locked	0、0	没有已记录/锁定错误位置。
cache RAM内容	不由 cm7icu 复位	在invalidate完成前不得作为有效cache内容使用。

Reset All Registers (RAR)参数为1时，地址、数据和诊断类非关键寄存器也由 reset_n 或 po_reset_n 清零；RAR为0时部分寄存器不复位以节省面积。无论RAR取值如何，所有外部valid/progress/ack/error信号必须在复位后呈现无事务状态，不能让未复位payload在valid为0时变成可观察行为。

CCR.IC 只是使用/分配使能，不是invalidate命令。拉低它不会清tag RAM，也不会自动清LFB；已经被BIU接受且标记为可分配的linefill可以继续完成两次allocation， icu_dpu_lf_in_prog_o 在尾部完成前保持进行中。新的fetch不再做普通lookup，也不会建立新的可分配line。之后重新使能时，原有valid tag仍可能命中，所以软件若在cache关闭期间修改了对应instruction memory，必须在重新使能前执行适当的invalidate与ISB。

4. 外部接口

4.0 时钟、复位与配置参数

端口/参数	方向	语义与重实现要求
clk	input	普通pipeline、LFB、维护、ECC和MBIST状态都在上升沿更新。
reset_n	input	core功能复位，低有效；清所有外部可见valid、progress和状态机状态。
po_reset_n	input	power-on reset域，低有效；复位IEBR和系统错误记录等状态。
dftramhold_i	input	Design For Test (DFT，面向可测试性设计) RAM hold；为1时屏蔽tag/data RAM写使能。
ICACHE	parameter	为0时移除cache lookup/allocation/ECC RAM行为，RAM输出tie-off，维护请求立即ack。
CACHEECC	parameter	为0时移除ECC checker、IEBR自动错误路径和ECC输出。
RAR	parameter	控制非关键寄存器是否参与复位；不改变外部事务复位语义。

4.1 PFU取指接口

Signal	Direction	Width	Valid / timing contract	Reimplementation requirement
<code>pfu_icu_req_i</code>	PFU -> ICU	1	always valid	PFU 请求一个 64-bit doubleword fetch。
<code>pfu_icu_addr_i</code>	PFU -> ICU	[31:3]	valid when request	必须 doubleword aligned；[4:3] 选择 line 内 doubleword。
<code>pfu_icu_first_i</code>	PFU -> ICU	1	valid when request	reset 后第一笔、跨 cache line、force 后或 PFU 无法保证顺序 way 信息时为 1。
<code>pfu_icu_priv_i</code>	PFU -> ICU	1	valid when request	BIU request privilege 输入；cacheable fetch 会被 ICU 强制标为 privileged。
<code>pfu_icu_vf_i</code>	PFU -> ICU	1	valid when request	vector fetch sideband，透传到 BIU。
<code>icu_pfu_ack_o</code>	ICU -> PFU	1	valid with request	等于 request 且 lc2 not stalled；ack 表示 ICU 接管该 fetch。
<code>pfu_icu_attr_i</code>	PFU -> ICU	[3:0]	valid one cycle after request	memory attribute；bit [1] 为 cacheable 判定。ICU 在 first cycle 捕获并保存。
<code>icu_pfu_dvalid_o</code>	ICU -> PFU	1	always valid	本周期 <code>icu_pfu_data_o</code> 有效；PFU 侧必须已有 in-flight fetch，除非 formal/testbench 非法激励。
<code>icu_pfu_data_o</code>	ICU -> PFU	[63:0]	valid when dvalid	64-bit instruction data；fake hit 时 data 不可使用，下一拍 ECC error 指示 replay。
<code>pfu_icu_cancel_resp_i</code>	PFU -> ICU	1	always valid	取消 outstanding response；ICU 清 lc2 fetch，并 mask 已取消 single transaction 的返回数据。
<code>icu_pfu_ecc_err_o</code>	ICU -> PFU	1	always valid	对上一拍返回的 data 报告 ECC/fake-hit replay 条件；不能对 canceled 或 bus-faulted fetch 报告。
<code>icu_pfu_bus_err_o</code>	ICU -> PFU	1	valid when dvalid	BIU data beat 同周期 bus fault；不能由 cache hit 产生。

PFU/ICU 协议的一条关键限制是：若已有 fetch in flight，且没有 data valid/cancel，本周期不能 ack 新 fetch。返回 data 的同一周期可以 ack 下一笔 request，从而支持 hit path 每 cycle throughput。

4.2 BIU读接口

Signal	Direction	Width	Meaning
<code>icu_biu_req_ic2_o</code>	ICU -> BIU	1	lc2 miss 发出的 read request。
<code>icu_biu_addr_ic2_o</code>	ICU -> BIU	[31:3]	request address，通常为 critical doubleword address。
<code>icu_biu_single_ic2_o</code>	ICU -> BIU	1	1 表示只期望 1 beat；0 表示 linefill/non-cacheable burst。
<code>icu_biu_attr_ic2_o</code>	ICU -> BIU	[3:0]	通常透传memory attributes。cache关闭、该fetch进入lc2时没有lookup能力、或原属性non-cacheable时清cacheable bit；仅因两个way都被IEBR阻塞而禁止allocation时仍保留cacheable bit，使BIU返回完整4拍。
<code>icu_biu_priv_ic2_o</code>	ICU -> BIU	1	<code>fe_priv</code> 或 cacheable；cacheable instruction fetch 必须按 privileged 发给 BIU。
<code>icu_biu_vf_ic2_o</code>	ICU -> BIU	1	vector fetch sideband。
<code>biu_icu_ack_i</code>	BIU -> ICU	1	request accepted；ack 不应在 request 为 0 时出现。

Signal	Direction	Width	Meaning
<code>biu_icu_data_valid_i</code>	BIU -> ICU	1	BIU read data beat valid; 不能和 request start 同周期返回。
<code>biu_icu_data_i</code>	BIU -> ICU	[63:0]	64-bit beat。
<code>biu_icu_data_last_i</code>	BIU -> ICU	1	data beat 是当前 request 最后一个 beat。
<code>biu_icu_fault_i</code>	BIU -> ICU	1	data beat 同周期 bus fault。

BIU beat length contract: single request 返回 1 beat; cacheable request 返回 4 beat; non-cacheable request 返回从 critical doubleword 到 32-byte line boundary 的 1 到 4 beat。ICU 不允许在 BIU busy、masked transaction 未结束、LFB allocation 未完成时启动新 linefill。

4.3 I-cache RAM接口

RAM path	Signal group	Contract
Tag RAM	<code>icu_ram_tag_en_o[1:0]</code> 、 <code>icu_ram_tag_wr_o</code> 、 <code>icu_ram_tag_addr_o[9:0]</code> 、 <code>icu_ram_tag_wdata_o[28:0]</code> 、 <code>ram_icu_tag_rdata*_i[28:0]</code>	two way/bank enable; read data 在 enable 后一拍有效; tag layout { <code>ECC[6:0]</code> , <code>valid</code> , <code>tag[20:0]</code> }; write 时至少一个 enable 必须为 1。
Data RAM	<code>icu_ram_data_en_o[1:0]</code> 、 <code>icu_ram_data_wr_o</code> 、 <code>icu_ram_data_addr*_o[11:0]</code> 、 <code>icu_ram_data_wdata*_o[71:0]</code> 、 <code>ram_icu_data_rdata*_i[71:0]</code>	two data bank enable; data layout { <code>ECC[7:0]</code> , <code>data[63:0]</code> }; read validity 由 ICU 内部 read-valid tracking 和 tag hit 判定。
Cache size	<code>ram_icu_cache_size_i[3:0]</code>	legal values为 4KB、8KB、16KB、32KB、64KB; reset 后不应变化。ICU 用它生成 index/tag mask。
Test hold	<code>dftramhold_i</code>	为 1 时屏蔽 tag/data write enable, 用于 Design For Test (DFT) hold。

4.4 控制、维护、错误与MBIST接口

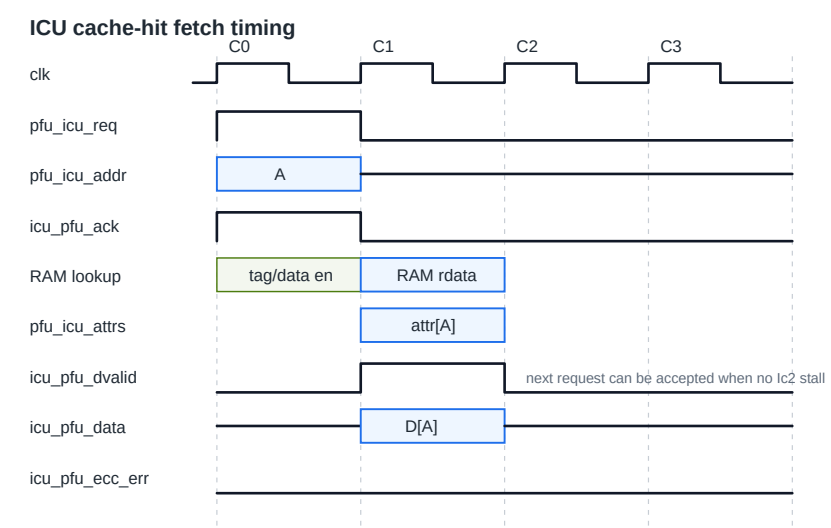
Interface	Signals	Behavior
Cache enable / ECC control	<code>dpu_icu_ccr_icen_i</code> 、 <code>dpu_icu_cacr_ecc_i</code>	<code>ccr_icen</code> 控制 lookup/allocation; <code>cacr_ecc</code> 在当前 RTL 中为 1 时屏蔽 ECC syndrome qualification。
Cache maintenance	<code>dpu_icu_cm_req_wr_i</code> 、 <code>dpu_icu_cm_ia_wr_i</code> 、 <code>dpu_icu_cm_addr_wr_i</code> 、 <code>icu_dpu_cm_ack_wr_o</code> 、 <code>icu_lsu_cm_in_prog_o</code>	request/ack 提交 cache maintenance; <code>cm_ia=0</code> 是 invalidate by MVA, <code>cm_ia=1</code> 是 invalidate all。
ISB	<code>dpu_icu_isb_wr_i</code>	ISB retire 使 LFB invalid, 避免旧 non-cacheable or stale linefill data 在 ISB 后被复用。
Linefill progress	<code>icu_dpu_lf_in_prog_o</code>	表示 ICU 内有 fetch、linefill、single read、unallocated LFB 或 masked BIU transaction。

Interface	Signals	Behavior
EBR PPB access	<code>dpu_icu_ebr_wr_i</code> 、 <code>dpu_icu_ebr_sel_i</code> 、 <code>dpu_icu_ebr_wdata_i</code> 、 <code>icu_dpu_ebr_rdata_o</code>	软件可选择并读写 EBR0/EBR1；write data 的 location/fatal/bank/way 字段只在 valid=1 时有意义。
External error report	<code>icu_ext_icerr_o[21:0]</code> 、 <code>icu_ext_icdet_o[3:0]</code>	向 system/PPB 报告 I-cache ECC error allocation、locked/full 情况和 error type 分类。
MBIST	<code>miu_icu_lock_req_i</code> 、 <code>icu_miu_lock_ack_o</code> 、 <code>miu_icu_rd_en_i</code> 、 <code>miu_icu_wr_en_i</code> 、 <code>miu_icu_data_en_i</code> 、 <code>miu_icu_mbistall_en_i</code> 、 <code>miu_icu_addr_i</code> 、 <code>miu_icu_banksel_i</code> 、 <code>miu_icu_wdata_i</code> 、 <code>icu_miu_rdata_o</code> 、 <code>icu_miu_err_o</code>	four-state lock handshake；lock 后 MIU 可读写 selected tag/data bank；read data 两拍后返回；read/write 不能同拍同时为 1。

5. 关键时序

所有timing图都以 `clk` rising edge为采样边沿；每条竖向cycle虚线都与一个 `clk` 上升沿重合，所有同步信号、状态框和数据框的左/右边界只能落在这些上升沿列。可变延迟用 `...` 表示，不代表固定拍数。

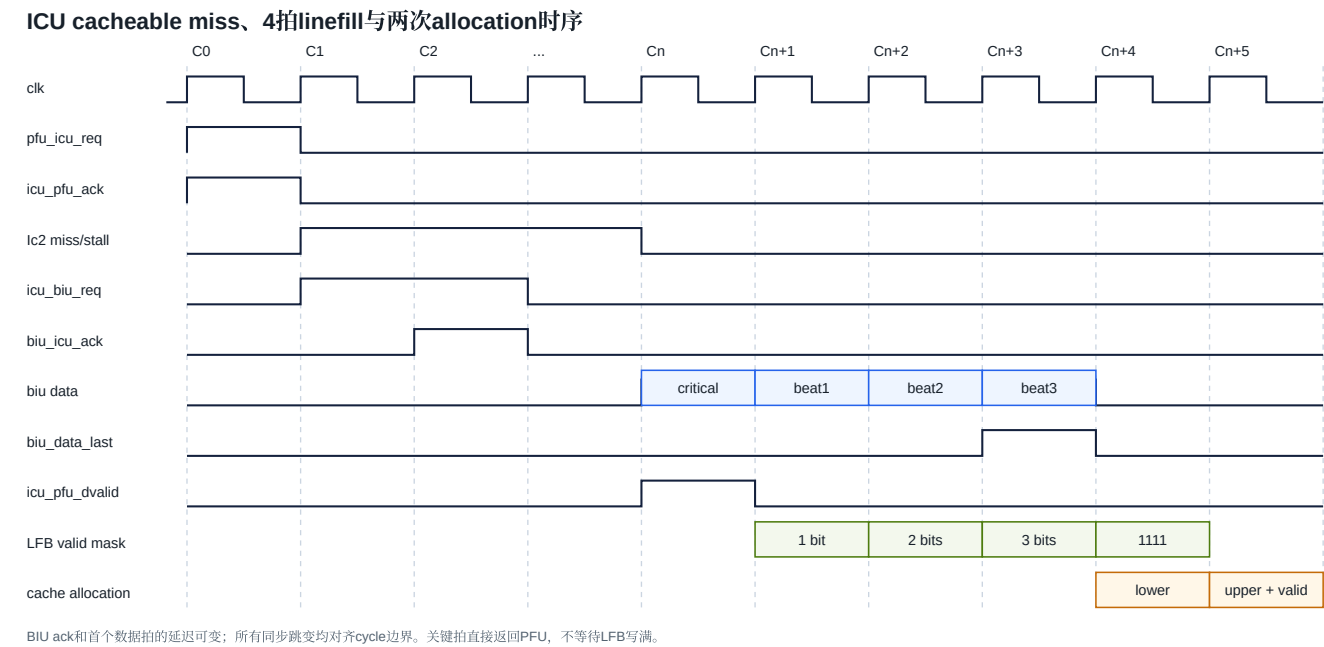
5.1 Cache hit时序



C0 中 PFU 发出 request，若 Ic2 没有 stall，ICU 同周期 ack 并在 Ic1 发起 RAM lookup。Tag/data RAM 是一拍读延迟，所以 C1 中 RAM rdata 到达 Ic2，同时 PFU 提供上一拍 request 对应的 attributes。若 tag hit 且 data 没有被 LFB/BIU/fake hit 覆盖，ICU 在 C1 输出 `icu_pfu_dvalid_o` 和 64-bit `icu_pfu_data_o`。如果没有 miss、maintenance、ECC invalidate 或 MBIST lock 造成 stall，C1 同时可以 ack 下一笔 PFU request。

本图省略 ECC error pulse，因为正常 hit 没有 ECC error。若同一 RAM read 检测到 data/tag ECC，data valid 仍可能先出现，PFU 在下一拍根据 `icu_pfu_ecc_err_o` replay。

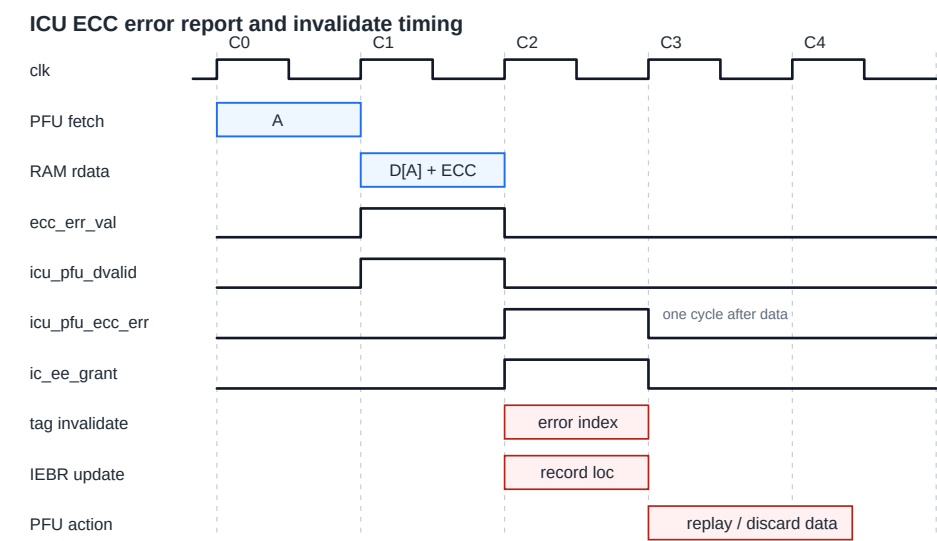
5.2 Miss、linefill与allocation时序



C1 中lc2判断miss后，如果BIU idle且LFB可用，ICU发起BIU request；BIU ack拍数可变，图中以 C2 ack为例。首个data beat延迟同样可变，但它必须是critical doubleword；ICU在 Cn 直接把该拍forward给PFU并输出 dvalid。四个cacheable beat按line内双字位置循环填满LFB，最后一拍带 data_last。四位valid在最后一拍采样后变为 1111，随后lower和upper各占一次RAM allocation grant，第二次写入才置tag valid。

如果BIU返回fault，fault beat同周期可返回给PFU并invalidate LFB。ISB、同line MVA或invalidate-all使正在linefill的LFB失效时，后续beat会被mask到 data_last。PFU cancel本身只清lc2响应：若在途的是single read，则屏蔽其返回；若在途的是普通linefill，linefill继续填充LFB并可完成cache allocation，只是不再把数据返回给已取消的fetch。

5.3 ECC错误时序



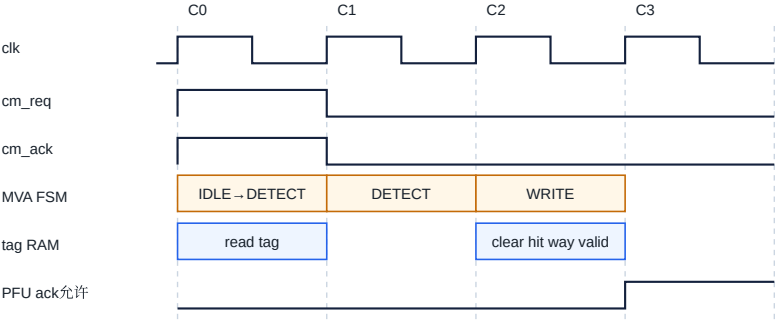
ECC syndrome与RAM rdata同拍组合计算。若 C1 的RAM read data被判定有ECC error，ICU注册错误分类和位置。PFU可在 C1 先看到 dvalid；icu_pfu_ecc_err_o 在 C2 报告，使PFU丢弃上一拍data并replay。C2 同时发起ECC invalidate，把对应index的两路tag都写成invalid，并更新IEBR。若两个IEBR都locked，system error report表示错误无法分配，但两路tag失效仍执行。

如果 C2 还有新的 lookup 读到与 ECC invalidate 相关的不可信 RAM data，ICU 可产生 fake hit：先给 PFU 一个 data-valid 事件，再在下一拍用 ECC error 标记 replay。fake hit 的 data 本身没有可实现语义，兼容实现只需保证 PFU 不会提交它。

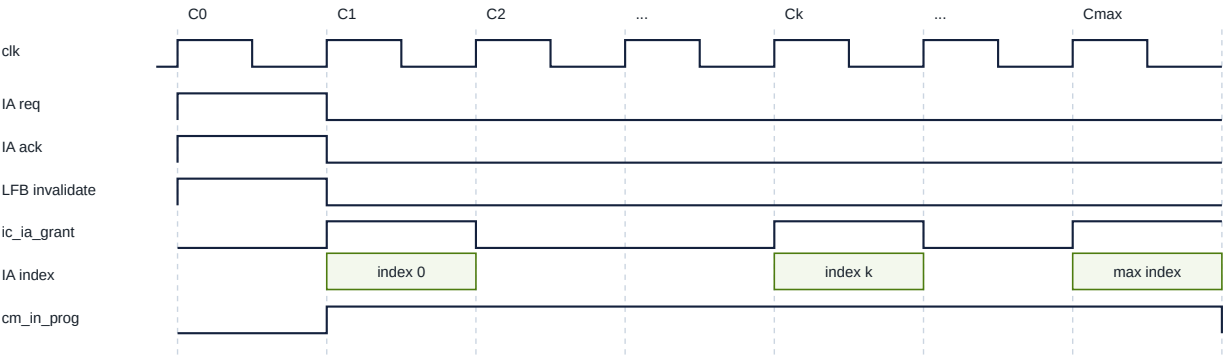
5.4 Cache维护时序

ICU cache maintenance关键时序

A. Invalidate by MVA: 立即获得tag端口且命中



B. Invalidate all: 后台低优先级遍历

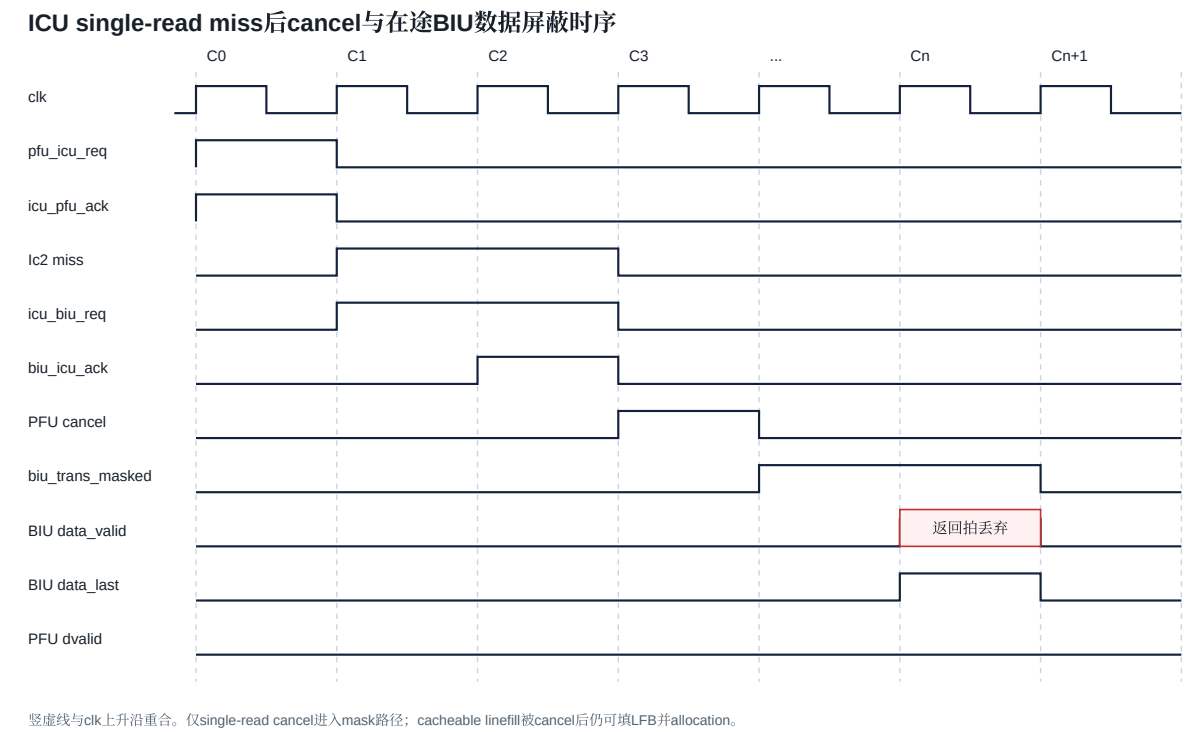


两部分中每条竖虚线都与clk上升沿重合；所有同步状态和信号只在这些边界变化。

Invalidate by MVA 是前台操作：request 被 ack 后，ICU 捕获 MVA 并执行 tag lookup；下一拍 DETECT 判断 tag hit；若命中，再请求 tag write 清除 valid。只要 MVA FSM 非 idle，ICU 会通过 stall 阻止新的 PFU fetch 进入 Ic2，以保证同一 index/line 的维护不会与 fetch 结果交错。若 MVA 只命中 LFB 而没有命中 tag，也必须 invalidate LFB 后完成。

Invalidate all 是后台操作：request 可以立即 ack，随后如果 cache dirty，ICU 低优先级遍历 tag index。图中 ic_ia_grant 可以被 lookup、MVA、ECC、allocation 或 MBIST 打断；因此 invalidate all 的完成时间是可变的。icu_lsu_cm_in_prog_o 在 invalidate all 背景遍历或 MVA FSM 非 idle 时有效，用于通知其他 core side 逻辑 maintenance 仍在进行。

5.5 Single read cancel与旧BIU事务屏蔽时序



图中cache lookup关闭且MIU lock存在，ICU因此发出single read；该请求已被BIU ack后，PFU在 `c3` 取消响应。ICU清当前`lc2 fetch`和 `single_in_prog`，不再输出 `dvalid`，同时置 `biu_trans_masked`。BIU协议不能被中途撤销，所以返回拍仍可能到达，但被丢弃到 `data_last`，之后才释放BIU busy。

普通cacheable linefill的cancel行为不同：只取消当前PFU响应，LFB仍接收剩余数据并可把完整line分配到cache，相当于保留已经发出的预取价值。只有ISB、同line MVA、invalidate-all或fault真正使LFB失效时，linefill后续拍才进入mask路径。同步信号只在cycle边界改变；从cancel或失效到 `data_last` 的间隔由BIU决定。

6. 重实现合同

6.1 实现目标

重写实现必须保持以下边界兼容：

- `cm7icu` 所有外部端口的 direction、width、valid timing 和 reset visible behavior。
- PFU fetch request/ack/data-valid/cancel/ECC/bus-error 协议。
- BIU request/ack/data beat/last/fault 协议和 beat length 语义。
- I-cache RAM 一拍 read latency、tag/data entry format、cache size mask、corkscrew bank mapping。
- Cache maintenance ack/completion/in-progress 语义。
- ECC error classification、IEBR update/mask/block、external ICERR/ICDET reporting。
- MBIST lock/read/write handshake 和 RAM arbitration priority。

内部实现可以改变物理RAM wrapper、状态编码、LFB寄存器packing和ECC生成器的门级结构，但必须保持容量mask、替换way选择条件、corkscrew、RAM一拍读延迟以及外部可见timing/priority/error/cancel/replay行为。若目标是cycle-accurate重实现，准随机默认way必须继续随PFU request翻转；只有不比较cache内部命中轨迹的架构级功能模型才可替换随机源。

6.2 必须保存的状态语义

State	Reset value	Update rules
fe_val_ic2	0	no stall 时变成当前 pfu_icu_req_i；stall 时 data valid 或 cancel 清 0，否则保持。
fe_addr_ic2、fe_priv_ic2、fe_vf_ic2	0 when RAR reset active	no stall 且 request 时捕获 PFU address/context。
fe_first_cycle_ic2	0	accepted request 后根据 PFU first、force-next-first、force-current-first 置 1；否则清 0。
pfu_icu_attrs_reg	0 when RAR reset active	first cycle 捕获 PFU attributes，后续 sequential fetch 复用。
lf_in_prog	0	linefill request ack 置 1；BIU data last 或 LFB invalidation 清 0。
single_in_prog_ic2	0	single request ack 置 1；PFU cancel 或 BIU data valid 清 0。
lfb_data_dword_val_ic2	0000	new linefill ack、LFB invalidation、MBIST access 清 0；linefill beat valid 时置对应 dword bit。
lfb_allocated_ic2	11	new allocatable linefill ack 置 00；每次 allocation grant 置 lower/upper bit；fault/invalidate/ECC block 置 11。
biu_trans_masked_ic2	0	outstanding linefill invalidated 或 single canceled 时置 1；BIU data last 清 0。
cm_im_state	STATE_IDLE	按3.8节状态转移表和状态跳转图转移。
cm_ia_val / cm_ia_addr	0 / 0	invalidate all start 且 cache dirty 后置 valid，从 index 0 递增到 masked max。
ic_dirty	1	任一 allocation grant 置 1；invalidate-all start 清 0；保证复位后第一次 IA 遍历。
default_alloc_way0	0	每当 pfu_icu_req_i=1 时翻转；只在两 way valid 相同且均未阻塞时作为默认选择。
tag0_hit_ic2_reg / rdata_from_ic_ic2_reg	0	first fetch 完成时记录顺序流命中 way 及是否真正来自 cache，供 small/reduced lookup 使用。
iebr0 / iebr1	invalid/unlocked/zero fields	PPB write 覆盖 selected register；ECC auto allocation 写 unlocked target register。
MBIST pipeline mb0/mb1/mb2	0	lock ack 或 MBISTALL 使 mb0 active；mb1/mb2 为 RAM access/read response pipeline。

RAR 为 0 时，部分 data/context register 不受 reset 清零；重写若用于 cycle-accurate RTL 兼容，应按 RAR 参数区分 reset domain。若只做功能模型，可在 reset 时清零全部内部状态，但必须保证 reset 后外部 outputs 不违反接口约束。

6.3 必须保持的优先级与选择规则

RAM grant priority 必须为：

Priority	Request	Grant name	Notes
1	MBIST	ic_mb_grant	MBIST 进入时独占 RAM；不应与 ECC invalidate 同时竞争。
2	ECC invalidate	ic_ee_grant	对出错 index 的两路 tag 写 invalid。
3	invalidate by MVA	ic_im_grant	MVA lookup/write 前台操作。

Priority	Request	Grant name	Notes
4	normal lookup	ic_lu_grant	Ic1 fetch lookup。
5	LFB allocation	ic_al_grant	写 data/tag RAM。
6	invalidate all	ic_ia_grant	background walker，最低优先级。

PFU 返回数据优先级必须为：

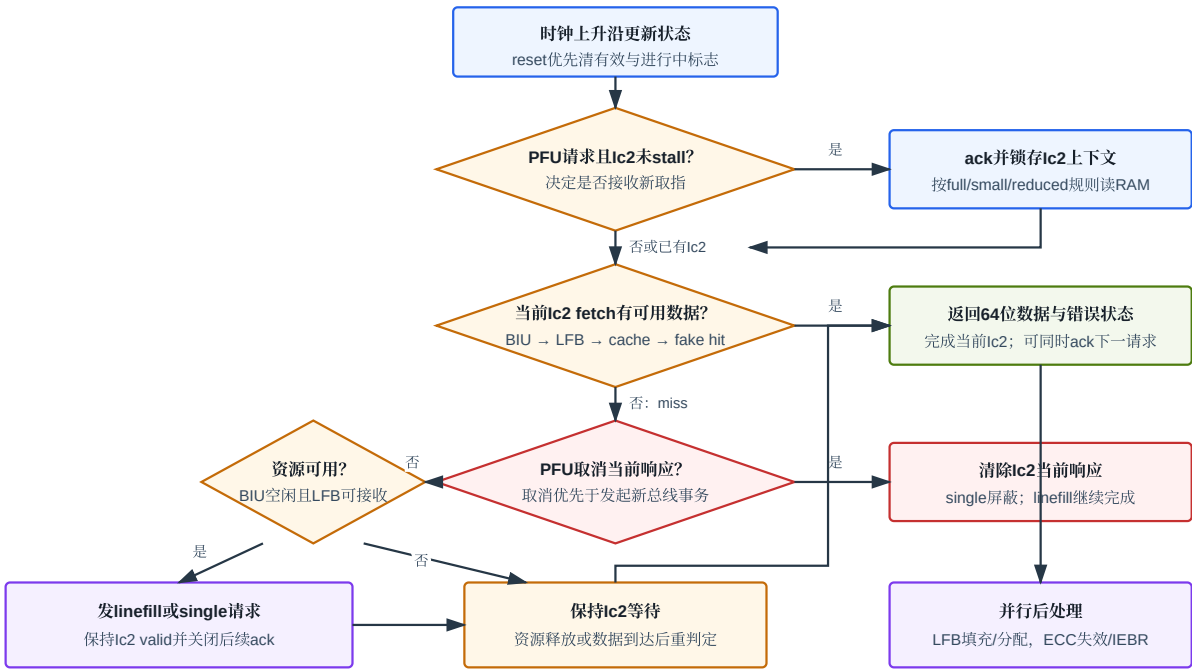
- 1. BIU 当前返回的数据拍。
- 2. LFB 中地址匹配的 64-bit doubleword。
- 3. fake hit占位数据；它必须屏蔽不可信cache data。
- 4. I-cache选中的数据 bank。
- 5. 无可用数据，保持 Ic2 未完成。

停顿/应答要求：

icu_pfu_ack_o 只在 pfu_icu_req_i=1 且 icu_stall_ic2=0 时为1。icu_stall_ic2 覆盖当前Ic2 miss、MVA FSM非空闲、MVA/ECC RAM grant、cache使能时的MBIST grant，以及cache使能时的MIU lock。cycle-accurate重实现不得额外加入可观察stall；架构级模型可以更保守，但必须保持前向进展，并明确不用于性能/时序等价比较。

6.4 必需算法

cm7icu取指主算法流程



流程图只画主判断链；精确的stall、替换、分配、ECC与维护优先级由正文表格定义。

这张流程图把6.4的文字算法压缩成主判断链：每个 `clk` 上升沿先处理reset；如果PFU request可以被接收，ICU把它变成下一拍Ic2 fetch context，并按lookup policy发起RAM查找。Ic2有效后先按BIU、LFB、fake hit、I-cache优先级寻找数据；有数据就返回PFU，否则进入miss/cancel分支。PFU cancel清Ic2；single事务需要mask旧返回，而普通linefill继续完成。未cancel且资源可用时发linefill或single请求并保持Ic2 stall。紫色框表示并行后处理，细节仍由后续文字算法定义。

取指命中/未命中算法：

每个 clk 上升沿：

如果 reset 有效：

清除对外可见的有效位和进行中状态

否则：

如果 PFU 有请求且 Ic2 没有 stall：

接收该请求，并作为下一拍 Ic2 取指上下文

按查找策略决定是否在 Ic1 发起 RAM 查找

如果 Ic2 有效：

根据属性计算本次取指是否可缓存

根据有效 tag、带屏蔽的 tag 比较、ECC/IEBR 屏蔽结果计算 tag 命中

根据 line address 和 doubleword 有效位计算 LFB 命中

根据当前 BIU linefill/single 数据拍计算 BIU 命中

如果 BIU 命中、LFB 命中、cache 命中或 fake hit 任一成立：

驱动 `icu\_pfu\_dvalid\_o` 和选中的 `icu\_pfu\_data\_o`

清除 Ic2 有效位；如果 replay/error 路径需要内部上下文，可仅保留内部诊断状态

否则如果没有 cancel 且 BIU/LFB 资源可用：

发起 BIU linefill 请求或 single 请求

保持 Ic2 有效，并 stall 后续 PFU 请求

否则如果 PFU 已 cancel：

清除 Ic2 有效位

如果是未完成 single transaction：屏蔽其返回直到 data_last

如果是普通 linefill：继续填充/分配，但不再向该 fetch 返回 data

Cache 分配算法：

当 BIU linefill 请求被 ack：

记录 line address 和选中的分配 way

如果本次取指不允许分配到 cache：

将 lower/upper 两个 qword 都标记为已分配

否则：

将 lower/upper 两个 qword 都标记为未分配

对每个未被屏蔽的 BIU data beat：

按 linefill address[4:3] 选择 LFB doubleword slot，并写入该数据拍

置位对应有有效位，并把 doubleword index 加 1

当 LFB 四个有效位全部置位：

如果 lower qword 尚未分配，且没有 invalidate/block：

将 lower qword 的两个 doubleword 写入两个 data bank

同时写 tag，但 tag valid bit 写 0

标记 lower qword 已分配

否则如果 upper qword 尚未分配，且没有 invalidate/block：

将 upper qword 的两个 doubleword 写入两个 data bank

同时写 tag，并把 tag valid bit 写 1

标记 upper qword 已分配

ECC/IEBR 处理算法：

对每个具有有效读使能历史的 RAM read:

如果 ECC 没有被控制信号、IEBR、fake hit 或无效 tag 屏蔽:

计算 tag/data ECC syndrome

如果任一 syndrome 非 0:

按优先级选择并注册一个 error: tag 优先于 data, fatal 优先于 correctable, way0 优先于 way1

按 cache size 和 tag/data bank 类型屏蔽后注册 error location

下一拍:

对选中的 index 发起 ECC invalidate, 同时清除两个 tag way

如果存在未 locked 的 IEBR, 则分配一个 IEBR entry

更新 ICERR/ICDET

如果该 error 影响了已返回给 PFU 的 fetch data, 则在 dvalid 后一拍报告 PFU ECC error

6.5 硬件抽象边界

可替换组件:

- ECC generator/checker implementation, 只要 syndrome classification、fatal detection、ICDET/ICERR/IEBR behavior 等价。
- RAM macro implementation, 只要保持一拍 read latency、write mask、entry layout、cache size mask 和 bank mapping。
- 准随机源在架构级模型中可替换; cycle-accurate RTL必须保留“PFU request时翻转、复位偏向way1”的更新规则。
- State encoding, 只要外部 timing/priority 不变。

不可替换或必须精确兼容的组件/行为:

- PFU/BIU/RAM/MIU/PPB/DPU-visible port protocol。
- LFB invalidation、BIU transaction mask、PFU cancel 与 ECC report suppression。
- `ccr_icen`、`cacr_ecc`、cacheability attribute 对 lookup/allocation/BIU attr 的影响。
- Cache maintenance ack/in-progress/foreground-vs-background 区分。
- IEBR register field layout和 software write mask。

6.6 验证矩阵

测试项	激励	期望结果	覆盖点
reset 后 first fetch 命中	释放 reset, PFU 发 first request, RAM 中预置命中的 tag/data	同周期 ack, 下一拍 dvalid/data, 无 bus/ecc error	PFU 协议、full lookup、RAM 读延迟
复位后首次启用	RAM预置任意旧valid, 保持 <code>CCR.IC=0</code> , 执行IA后再置1	cache关闭期间不lookup; IA遍历全部有效set; 启用后旧 line不能命中	reset、 <code>ic_dirty=1</code> 、首次 IA
五种容量地址mask	依次配置4/8/16/32/64 KiB, 访问边界 index与同index异tag地址	set数量分别为64/128/256/512/1024, tag/index边界正确, 无地址别名误命中	cache-size mask
顺序 reduced lookup	first fetch 命中 way0/way1 后, 连续发同一 cache line 内的 fetch	后续 lookup 使用 reduced/small policy, 并返回正确的 corkscrew bank data	first/reduced lookup、bank 映射
cache miss linefill	PFU 发 cacheable miss, BIU 返回 4 个数据拍	critical 数据拍 forward 给 PFU, LFB valid mask 填满, 先 lower 后 upper allocation, tag 只在 upper allocation 后有效	miss、LFB、allocation 顺序
partial-LFB forward	从DW2开始linefill, 只返回DW2后暂停后续beat; 随后PFU请求同line的 DW2	valid mask仅为 0100 时仍从LFB返回DW2, 不等待其他槽; 此时不得发起cache allocation	单DW有效返回、stored-LFB hit、完整line分配门槛

测试项	激励	期望结果	覆盖点
non-cacheable miss	attributes 中 cacheable bit 为 0，BIU 返回 1 到 4 个数据拍	data 通过 BIU/LFB forward，不产生有效 cache allocation，BIU attrs 的 cacheable bit 被清除	attribute 处理、no-allocation
替换唯一无效way	目标set一路valid、一路invalid，发 cacheable miss	选择invalid way，与准随机位无关	replacement valid优先
替换way被IEBR阻塞	对目标index阻塞way0或way1，再发 miss	只选未阻塞way；两路均阻塞时仍返回数据但不allocation	IEBR block、no-allocation
linefill期间PFU cancel	cacheable linefill被BIU接受后PFU cancel，随后返回剩余4拍	lc2清除，不为canceled fetch输出dvalid/ecc；linefill继续填LFB并可allocation	cancel保留预取、no response
single read期间PFU cancel	MIU lock/cache lookup off路径的 single request被接受后cancel	清 single_in_prog，后续返回被mask到last，不写LFB 且不输出dvalid	cancel、BIU mask
BIU bus fault	miss 后 BIU data_valid 同拍带 fault	同周期输出 dvalid 和 bus_err，LFB invalidated，同一 fetch 不报告 ECC error	bus error path
cache hit data ECC error	cache hit 的 data 带 syndrome	先dvalid，下一拍ecc_err，IEBR被分配，出错index两路 tag都失效，ICDET data bit置位	ECC timing、IEBR、PFU replay
tag index field ECC error	tag syndrome 指向 index field	判为 fatal，报告 tag bank，location 低位按 tag error 规则 mask	fatal tag policy
IEBR locked/full	两个 IEBR 都 locked 后再注入 ECC error	不自动覆盖 EBR，ICERR 指示无法分配或 locked 状态，后续 allocation 只受已有 valid entry block	EBR policy
invalidate by MVA hit	PPB 对 cached line 发 MVA request	仅 FSM idle 时 ack，tag lookup 后清 tag valid，匹配的 LFB invalidated	MVA FSM
invalidate by MVA miss 但 LFB match	MVA 不命中 tag，但 LFB 中有同 line valid data	不写 tag，LFB invalidated，FSM 返回 idle	LFB maintenance
invalidate all dirty	cache allocation 后发 IA request	立即 ack，后台从 index 0 到 max 清 tag valid；高优先级 RAM grant 可打断后台遍历	IA background
invalidate all被打断	IA进行中连续插入lookup/allocation	cm_in_prog 持续为1，index只在IA grant后加1，无set漏清	IA保持、RAM仲裁
MBIST lock and read	ICU idle 时 MIU 发 lock request，并读 tag/data bank	lock ack，RAM access，read data 两拍后返回，normal lookup 被阻止	MBIST
MBIST vs possible ECC	可能产生 ECC 的 RAM read pending，同时 MIU 发 lock request	lock ack 延后，直到 possible/registered ECC 清除	ECC/MBIST priority
cache disabled fetch	ccr_icen=0 且 attribute 不可缓存	不做 normal lookup/allocation，miss 走 BIU，PFU 收到 data	cache disable

7. 已知未知项

项目	状态
当前解包中的 cm7icu_decl.v 为空	TBD：只有精确 RTL 状态编码会因此受阻。功能状态名和状态转移已经能从 cm7icu.v 还原；若目标需要 netlist/scan/trace 级完全兼容，还需要原始 declaration source 来确认 STATE_IDLE、STATE_WAIT、STATE_DETECT、STATE_WRITE、STATE_LFB_INVAL 的具体编码。
dpu_icu_cacr_ecc_i 命名	PPB interface 注释把它描述为 cache ECC control，但 ICU 使用该信号的反相信号参与 syndrome qualification。重写必须以 RTL 行为为准：该信号为 1 时屏蔽 ICU 的 ECC 检测/上报。
cm7icu_ecc_check 输出的 pfu_ecc_err_val_o	ECC checker 生成该信号，但当前 cm7icu.v 顶层没有在可见输出或 stall 逻辑中使用它。除非其他 source revision 对它重新连线，否则按未使用的内部诊断信号处理。

项目	状态
Performance Monitoring Unit (PMU) event	interface spec 提到 PMU event 仍待添加；当前可读 source 没有定义 ICU 对 PMU 可见的 event 行为。